```
30            .password(userDto.getPassword())
31            .authorities(privilegeArray).build();
32        return userDetails;
33    }
34 }
```

在上述代码中，第 12～13 行代码注入 UserDao 对象；第 19 行代码根据用户名获取用户信息；第 20～23 行代码判断如果没有获取到用户信息，则返回 null，抛出异常；第 25 行代码获取用户对应的权限；第 29 行代码创建了一个 UserDetails 对象，将查询到的用户信息封装在该对象中并返回。

（4）效果测试

将文件 7-9 中的 userDetailsService()方法进行注释，使用自定义 UserDetailsService 身份认证。重启 chapter07 项目进行效果测试，项目启动成功后，在浏览器通过“ http://localhost:8080/ ”访问项目首页后，使用错误的数据库用户信息进行登录，例如使用用户名 :zhangsan，密码：1234，效果如图 7-13 所示。

从图 7-13 可以看出，使用错误的用户信息登录时，认证失败并在页面中提示“ 用户名或密码错误 ”。

在图 7-13 中使用正确的数据库用户信息进行登录，效果如图 7-14 所示。

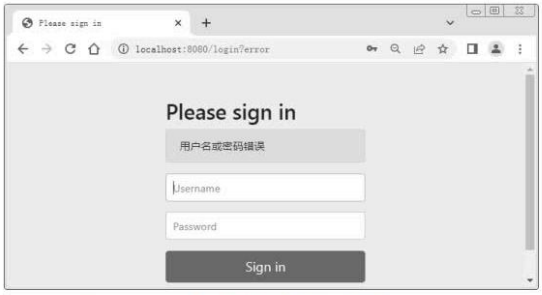


图7-13　使用错误用户信息登录（3）

图7-14　项目登录成功效果（3）

从图 7-14 可以得出，使用数据库中对应的用户信息进行登录后认证成功，说明通过自定义 UserDetailsService 身份认证方式实现了自定义身份认证。

至此，完成了 Spring Boot 整合 Spring Security 进行自定义身份认证的讲解。其中，内存身份认证最为简单，主要用作测试和新手体验；JDBC 身份认证和 UserDetailsService 身份认证在实际开发中较为常用，对于这两种认证方式，读者可以根据实际开发中已有业务自行选择。

7.4　Spring Security 授权管理

授权是 Spring Security 的核心功能之一，是根据用户的权限来控制用户访问资源的过程，拥有资源的访问权限时可正常访问，没有访问的权限时则会被拒绝访问。认证是为了保证用户身份的合法性，而授权则是为了更细粒度地对隐私数据进行划分，授权是在认证通过后发生的，以控制不同的用户访问不同的资源。Spring Security 提供了授权方法，开发者通过这些方法进行用户访问控制，下面对 Spring Security 的授权流程和自定义授权进行讲解。

7.4.1　Spring Security 授权流程

实现授权需要对用户的访问进行拦截校验，校验用户的权限是否可以操作指定的资源。Spring Security 使用标准 Filter 建立了对 Web 请求的拦截，最终实现对资源的授权访问。下面通过一张图展示 Spring Security 的授权流程，具体如图 7-15 所示。

下面结合图 7-15 对 Spring Security 的授权流程进行详细介绍。

拦截请求。已认证用户访问受保护的 Web 资源将被 SecurityFilterChain 中的 Filter SecurityInterceptor 实例对象拦截。

获取资源访问策略。FilterSecurityInterceptor 实例对象会通过 SecurityMetadataSource 的子类的 DefaultFilterInvocationSecurityMetadataSource 实例对象获取要访问当前资源所需要的权限，权限封装在 Collection 实例对象中。SecurityMetadataSource 用于提供安全对象的元数据信息，这些元数据通常包括用于访问控制决策所需的授权属性（如权限），Spring Security 中提供的 SecurityMetadataSource 实现类会根据应用程序的安全访问策略的规则来读取和返回这些元数据。

FilterSecurityInterceptor 通过 AccessDecisionManager 进行授权决策，若决策通过，则允许访问资源，否则将禁止访问。AccessDecisionManager 中包含一系列 AccessDecision Voter，可对当前认证过的身份是否有权访问对应的资源进行投票，AccessDecisionManager 根据投票结果做出最终决策。

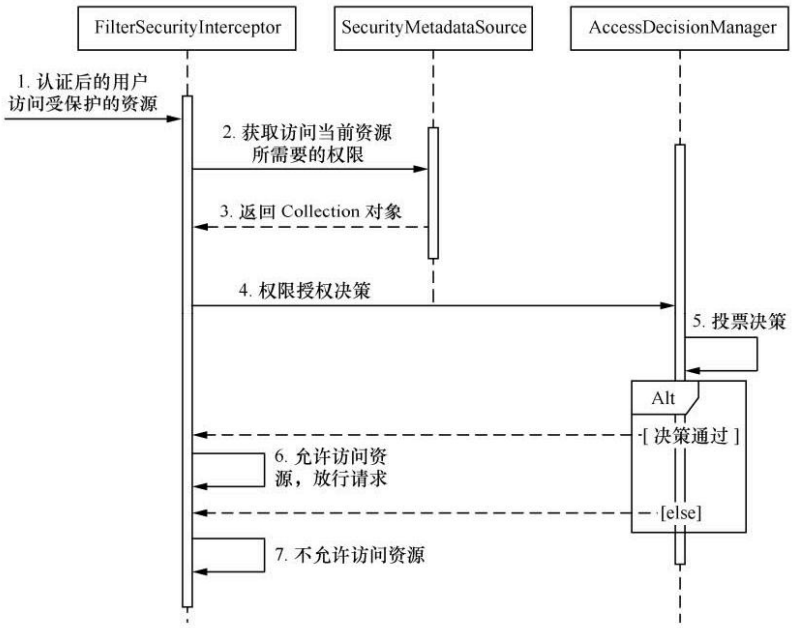


图7-15　Spring Security的授权流程

7.4.2　Spring Security 自定义授权

根据授权的位置和形式，通常可以将授权的方式分为 Web 授权和方法授权，两种授权方式都会调用 AccessDecisionManager 进行授权决策。下面分别对这两种自定义授权的方式进行讲解。

1. Web 授权

通过之前的学习可知，Spring Security 的底层实现本质是通过多个 Filter 形成的过滤器链完成的，过滤器链中提供了默认的安全拦截机制，设置了安全拦截规则，以控制用户的访问 ,例如指定 URL 匹配对应资源的权限、From 表单认证等。HttpSecurity 是 SecurityBuilder 接口的实现类，是 HTTP 安全相关的构建器，Spring Security 中可以通过 HttpSecurity 对象设置安全拦截规则，并通过该对象构建过滤器链。

HttpSecurity 可以根据不同的业务场景，针对不同的 URL 采用不同的权限处理策略。当开发者需要配置项目的安全拦截规则时，可以调用 HttpSecurity 对象对应的方法实现。HttpSecurity 类中常用的方法如表 7-1 所示。

表 7-1 HttpSecurity 类的常用方法

方法	作用
authorizeRequests()	开启基于 HttpServletRequest 请求访问的限制
formLogin()	开启基于表单的用户登录
httpBasic()	开启基于 HTTP 请求的 Basic 认证登录
logout()	开启退出登录的支持
sessionManagement()	开启 Session 管理配置
rememberMe()	开启“记住我”功能
csrf()	配置 CSRF 跨站请求伪造防护功能

上述方法列举了 HttpSecurity 类的常用方法，通常 Java 大部分 Web 应用中的请求都是基于 HttpServletRequest 请求的。通过 authorizeRequests()方法可以添加用户请求控制的规则，这些规则通过用户请求控制的相关方法指定，具体如表 7-2 所示。

表 7-2 用户请求控制的常用方法

方法	作用
antMatchers(String... antPatterns)	开启 Ant 风格的路径匹配
mvcMatchers(String... patterns)	开启 MVC 风格的路径匹配，与 Ant 风格类似
regexMatchers(String... regexPatterns)	开启正则表达式的路径匹配
and()	功能连接符
anyRequest()	匹配任何请求
rememberMe()	开启“记住我”功能
access(String attribute)	使用基于 SpEL 表达式的角色进行匹配
hasAnyRole(String... roles)	匹配用户是否有参数中的任意角色
hasRole(String role)	匹配用户是否有某一个角色
hasAnyAuthority(String... authorities)	匹配用户是否有参数中的任意权限
hasAuthority(String authority)	匹配用户是否有某一个权限
authenticated()	匹配已经登录认证的用户
fullyAuthenticated()	匹配完整登录认证的用户（非 rememberMe 登录用户）
hasIpAddress(String ipaddressExpression)	匹配某 IP 地址的访问请求
permitAll()	无条件对请求进行放行

表 7-2 列举了用户请求访问中涉及的主要方法和说明，关于更多的方法读者可以自行查询 API 文档。另外，表 7-2 中涉及了用户的角色 Role 和权限 Authority，在自定义用户访问控制时，通过角色 Role 相关的方法和权限 Authority 相关的方法都可以定义用户访问控制。

尽管 Spring Security 提供的默认登录页面很方便，可以快速启动和运行，但大多数应用程序都希望定义自己的登录页面。通过 HttpSecurity 类的 formLogin()方法开启基于表单的用户登录后，可以指定表单认证的相关设置，基于表单的身份验证的常见方法如表 7-3 所示。

表 7-3　基于表单的身份验证的常见方法

方法	作用
loginPage(String loginPage)	指定自定义登录界面，不使用 SpringSecurity 默认登录界面
loginProcessingUrl(String loginProcessingUrl)	指定处理登录的请求 URL，为表单提交用户信息的 Action
successForwardUrl(String forwardUrl)	指定登录成功后默认跳转的路径

下面通过案例演示在 Spring Boot 项目中使用 Spring Security 的 Web 授权方式进行权限管理，具体如下。

（1）导入登录页面

在项目 chapter07 的 resources 目录的 templates 文件夹中导入自定义的登录页面，读者可以自行创建，也可以直接使用本书配套资源中提供的页面，本书提供的登录页面的核心代码如文件 7-14 所示。

文件 7-14　login.html

```
<!-- ……页面其他代码  -->
<form id="loginform" class="sui-form" th:action="@{/doLogin}"  method="post">
    <div class="input-prepend"><span class="add-on loginname">用户名</span>
        <input type="text" placeholder="用户名" class="span2 input-xfat"
name="username">
    </div>
    <div class="input-prepend"><span class="add-on loginpwd">密码</span>
        <input type="password" placeholder="请输入密码"
              class="span2 input-xfat" name="password">
    </div>
    <div class="logined">
        <a class="sui-btn btn-block btn-xlarge btn-danger"
          href='javascript:document:loginform.submit();' target="_self">
         登  录
       </a>
    </div>
</form>
<!-- ……页面其他代码  -->
```

在上述代码中，第 2～16 行代码定义了一个表单，其中第 2 行代码指定了表单提交的路径为“ /doLogin ”，表单中用户名和密码输入框的 name 属性值必须为 username 和 password。

（2）编辑配置类

在项目 chapter07 的 WebSecurityConfig 配置类中使用 HttpSecurity 对象设置安全拦截规则，并创建 SecurityFilterChain 对象交由 Spring 管理，具体如文件 7-15 所示。

文件 7-15 WebSecurityConfig.java

```
 1  import org.springframework.context.annotation.Bean;
 2  import org.springframework.context.annotation.Configuration;
 3  import
org.springframework.security.config.annotation.web.builders.HttpSecurity;
 4  import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
 5  import org.springframework.security.crypto.password.PasswordEncoder;
 6  import org.springframework.security.web.SecurityFilterChain;
 7  @Configuration
 8  public class WebSecurityConfig  {
 9     @Bean
 10    public PasswordEncoder passwordEncoder() {
 11       return new BCryptPasswordEncoder();
 12    }
 13    @Bean
 14    public  SecurityFilterChain securityFilterChain(HttpSecurity http) throws
Exception{
 15          http.authorizeRequests() // 定义哪些 URL 需要被保护、哪些不需要被保护
 16                .mvcMatchers("/loginview","/css/**","/img/**").permitAll()
 17                .mvcMatchers("/book/admin/**").hasRole("ADMIN")
 18                .anyRequest().authenticated()  // 任何请求,登录后可以访问
 19             .and()
 20             .formLogin()
 21                .loginPage("/loginview")
 22                .loginProcessingUrl("/doLogin")
 23             .and()
 24             .csrf().disable()//禁止 CSRF 跨站请求保护;
 25             .headers().frameOptions().sameOrigin();
 26       return http.build();
 27    }
 28 }
```

在上述代码中，第 14～27 行代码通过 securityFilterChain()方法创建 SecurityFilterChain 对象交由 Spring 管理。第 15～18 行代码对基于 HttpServletRequest 的请求访问进行授权管理，其中第 16 行代码设置不用认证直接放行的请求路径；第 17 行代码指定认证成功后，需要拥有 ADMIN 权限的用户才能访问的资源路径，“ ** ”表示所有。

第 20～22 行代码对基于表单认证的资源进行管理，其中第 21 行代码指定登录页面，第 22 行代码指定表单提交路径，提交用户登录表单后，Spring Security 会捕获该请求路径中的 username 和 password 进行身份认证。

第 24 行代码用于屏蔽 CSRF 控制，即 Spring Security 不再限制跨站请求伪造。本案例的页面使用 iframe 框架嵌入内容展示页面，第 25 行代码设置页面可以在相同域名页面的 frame 中展示，如果不设置浏览器会阻止相同域名的页面嵌套。第 26 行代码用于根据 HttpSecurity 对象构建 SecurityFilterChain 对象并返回。

文件 7-15 中指定了登录页面的映射路径为 loginview ,访问登录页面请求只需直接跳转到登录页面即可，没有其他的业务逻辑，可直接在文件 7-5 的 WebMvcConfig 配置类中添加 loginview 的视图映射，具体如文件 7-16 所示。

文件 7-16　WebMvcConfig.java

```
1  import org.springframework.context.annotation.Configuration;
2  import
org.springframework.web.servlet.config.annotation.ViewControllerRegistry;
3  import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;
4  @Configuration
5  public class WebMvcConfig implements WebMvcConfigurer {
6      @Override
7      public void addViewControllers(ViewControllerRegistry registry) {
8          registry.addViewController("/").setViewName("/main.html");
9          registry.addViewController("/loginview").setViewName("/login.html");
10     }
11 }
```

在上述代码中，第 9 行代码将请求路径“ /loginview ”映射到视图“ /login.html ”。

（3）测试效果

启动项目 chapter07，在浏览器中通过“ http://localhost:8080/ ”访问项目首页，效果如图 7-16 所示。

从图 7-16 可以看出，当用户未进行身份认证访问项目中的资源时，自动跳转到自定义的登录页面，说明开启基于表单的用户登录的配置生效了。

在图 7-16 中，使用 zhangsan 的用户信息进行登录，效果如图 7-17 所示。

图7-16　登录页面

图7-17　使用zhangsan进行登录（1）

从图 7-17 可以看出，页面跳转到后台首页，说明使用 zhangsan 登录成功。

图书管理对应的请求为“ /book/admin/manag ”，需要角色为 ROLE_ADMIN 的用户才可以访问。用户 zhangsan 的角色为 ROLE_COMMON，在图 7-17 中单击“ 图书管理 ”链接，效果如图 7-18 所示。

从图 7-18 可以看出，内容展示页显示异常状态码为 403 的异常信息，说明当前用户没有权限访问该资源。

在图 7-16 中，使用用户 lisi 进行登录，lisi 对应的角色为 ROLE_ADMIN，登录成功再次访问“ 图书管理 ”，效果如图 7-19 所示。

图7-18　zhangsan访问“ 图书管理 ”（1）

图7-19　lisi访问“ 图书管理 ”（1）

从图 7-19 中可以看出，用户 lisi 成功访问“图书管理”，说明基于 HttpServletRequest 请求访问的授权设置成功。

2. 方法授权

Spring Security 除了可以在配置类中通过创建过滤器链设置安全拦截规则外，还可以使用@Secured、@RolesAllowed 和@PreAuthorize 注解控制类中所有方法或者单独某个方法的访问权限，以实现对访问进行授权管理。

使用@Secured 和@RolesAllowed 注解时，只需在注解中指定访问当前注解标注的类或方法所需要具有的角色，允许多个角色访问时，使用大括号对角色信息进行包裹，角色信息之间使用分号分隔即可，示例如下。

```
1  @RequestMapping("list")
2  @Secured({"ROLE_ADMIN","ROLE_COMMON"})
3  public String findList() {
4      return "book_list";
5  }
6  @RequestMapping("admin/manag")
7  @RolesAllowed("ROLE_ADMIN")
8  public String findManagList() {
9      return "book_manag";
10 }
```

在上述代码中，第 2 行代码使用@Secured 注解指定用户访问 findList()方法时需要具备 ROLE_ADMIN 或 ROLE_COMMON 的角色；第 7 行代码使用@RolesAllowed 注解指定用户访问 findManagList()方法时需要具备 ROLE_ADMIN 的角色。

@PreAuthorize 注解会在方法执行前进行权限验证，支持 SpEL 表达式，示例代码如下。

```
1  @RequestMapping("list")
2  @PreAuthorize("hasAnyRole('ROLE_ADMIN','ROLE_COMMON')")
3  public String findList() {
4      return "book_list";
5  }
6  @RequestMapping("admin/manag")
7  @PreAuthorize("hasRole('ROLE_ADMIN')")
8  public String findManagList() {
9      return "book_manag";
10 }
```

在上述代码中，第 2 行代码使用@PreAuthorize 注解指定用户访问 findList()方法时需要具备 ROLE_ADMIN 或 ROLE_COMMON 的角色；第 7 行代码使用@PreAuthorize 注解指定用户访问 findManagList()方法时需要具备 ROLE_ADMIN 的角色。

@Secured、@RolesAllowed 和@PreAuthorize 注解都可以对方法的访问进行权限控制，其中，@Secured 为 Spring Security 提供的注解；@RolesAllowed 为基于 JSR 250 规范的注解；@PreAuthorize 支持 SpEL 表达式。读者可以根据具体的需求自行选择注解。

Spring Security 默认是禁用方法级别的安全控制注解，要想使用注解进行方法授权，可以使用@EnableGlobalMethodSecurity 注解开启基于方法的安全认证机制，该注解可以标注在任意配置类上，示例如下。

```
@Configuration
@EnableGlobalMethodSecurity(securedEnabled = true,jsr250Enabled = true,
```

```
prePostEnabled = true)
public class WebSecurityConfig  {……}
```

在上述代码中，在配置类 WebSecurityConfig 上标注@EnableGlobalMethodSecurity 注解开启基于方法的安全认证机制，其中使用@Secured、@RolesAllowed 和@PreAuthorize 注解时，分别需要使用 securedEnabled、jsr250Enabled、prePostEnabled 属性指定值为 true。

下面通过案例演示在 Spring Boot 项目中使用@Secured 注解进行方法授权，具体如下。

（1）开启基于方法的安全认证机制

在项目 chapter07 的 WebSecurityConfig 配置类中开启基于方法的安全认证机制，并将类中 HttpSecurity 对象设置的访问“ 图书管理 ”拦截规则删除，以确保采用方法授权的方式对资源授权，修改后的代码如文件 7-17 所示。

文件 7-17　WebSecurityConfig.java

```
 1 import org.springframework.context.annotation.Bean;
 2 import org.springframework.context.annotation.Configuration;
 3 import org.springframework.security.config.annotation.method.configuration
 4           .EnableGlobalMethodSecurity;
 5 import
org.springframework.security.config.annotation.web.builders.HttpSecurity;
 6 import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
 7 import org.springframework.security.crypto.password.PasswordEncoder;
 8 import org.springframework.security.web.SecurityFilterChain;
 9 @Configuration
10 @EnableGlobalMethodSecurity(securedEnabled = true,prePostEnabled = true)
11 public class WebSecurityConfig  {
12    @Bean
13    public PasswordEncoder passwordEncoder() {
14       return new BCryptPasswordEncoder();
15    }
16    @Bean
17 public   SecurityFilterChain  securityFilterChain(HttpSecurity  http)  throws
Exception {
18         http.authorizeRequests() // 定义哪些 URL 需要被保护、哪些不需要被保护
19                  .mvcMatchers("/loginview","/css/**","/img/**").permitAll()
20                 .anyRequest().authenticated()  // 任何请求,登录后才可以访问
21             .and()
22             .formLogin()
23                 .loginPage("/loginview")
24                 .loginProcessingUrl("/doLogin")
25                 .permitAll()
26             .and()
27             .csrf().disable()//禁止 CSRF 跨站请求保护;
28             .headers().frameOptions().sameOrigin();
29     return http.build();
30 }}
```

在上述代码中，第 10 行代码使用@EnableGlobalMethodSecurity 注解在配置类上开启基于方法的安全认证机制。

（2）方法授权

在文件 7-4 BookController 类的 findManagList()方法上使用注解指定访问该方法所需的

角色，具体如文件 7-18 所示。

文件 7-18 BookController.java

```
1  import org.springframework.security.access.annotation.Secured;
2  import org.springframework.stereotype.Controller;
3  import org.springframework.web.bind.annotation.RequestMapping;
4  @Controller
5  @RequestMapping("book")
6  public class BookController {
7      @RequestMapping("list")
8      public String findList() {
9          return "book_list";
10     }
11     @RequestMapping("admin/manag")
12     @Secured("ROLE_ADMIN")
13     public String findManagList() {
14         return "book_manag";
15     }
16 }
```

在上述代码中，第 12 行代码使用@Secured 注解指定用户访问 findManagList()方法需要具有 ROLE_ADMIN 角色。如果将第 12 行代码标注在类上，则用户访问整个类中的所有方法都需要具有 ROLE_ADMIN 角色。

（3）测试效果

启动项目 chapter07，在浏览器中通过“ http://localhost:8080/ ”访问项目首页后，使用用户 zhangsan 登录系统，效果如图 7-20 所示。

从图 7-20 可以看出，页面跳转到后台首页，说明使用 zhangsan 登录成功。

单击图 7-20 中的“ 图书管理 ”链接，效果如图 7-21 所示。

图7-20 使用zhangsan进行登录（2）

图7-21 zhangsan访问“ 图书管理 ”（2）

从图 7-21 可以看出，内容展示页显示异常状态码为 403 的异常信息，说明当前用户没有权限访问该资源。

在图 7-20 中，使用用户 lisi 进行登录，登录成功后再次访问“ 图书管理 ”，效果如图 7-22 所示。

图7-22 lisi访问“ 图书管理 ”（2）

从图 7-22 中可以看出，用户 lisi 成功访问“ 图书管理 ”，说明使用注解进行方法授权与基于 HttpServletRequest 请求访问的授权设置效果一样。

7.4.3　动态展示菜单

在前面的讲解中，只是通过 Spring Security 对后台资源的访问根据角色进行权限控制，前端页面并没有做任何处理，不同角色能看到的前端页面是一样的，即使当前用户没有对应的访问权限，依然能看到对应的菜单，用户体验较差。灵活使用 Spring Security 的认证和授权可以实现多种需求，因此在开发过程中应勤于反思，善于总结经验，注重代码层面的优化，根据不同情境和自身实际，赋予应用程序更强的可扩展性和适应性，使应用程序具有更好的安全性和用户体验。

下面在前面案例的基础上 ,讲解如何使用 Spring Security 与 Thymeleaf 整合实现前端页面根据登录用户的角色动态展示菜单。

1. 添加依赖

Thymeleaf 提供了一个与 Spring Security 集成的附加模块 thymeleaf-extras-springsecurity，本书基于 Spring Security 5.7.5，案例演示之前需要在 chapter07 项目的 pom.xml 文件中添加 Thymeleaf 和 Spring Security 5 的集成包 thymeleaf-extras-springsecurity5，具体如下。

```
<dependency>
   <groupId>org.thymeleaf.extras</groupId>
   <artifactId>thymeleaf-extras-springsecurity5</artifactId>
</dependency>
```

上述添加依赖的版本号是由 Spring Boot 统一整合并管理的。

2. 修改页面代码

打开 chapter07 项目后台首页 main.html，引入 Spring Security 安全标签，并在页面中根据需求使用 Spring Security 标签指定为不同角色显示不同的页面内容，实现动态展示控制。修改文件 7-1，项目后台首页的核心内容如文件 7-19 所示。

文件 7-19　main.html

```
<html xmlns:th="http://www.thymeleaf.org"
      xmlns:sec="http://www.thymeleaf.org/extras/spring-security">
<html>
<head>
    <!--……静态资源导入代码，以及页面头部代码-->
   <!-- 导航侧栏 -->
   <aside class="main-sidebar">
      <section class="sidebar">
         <ul class="sidebar-menu">
           <li sec:authorize="hasAnyAuthority('ROLE_COMMON','ROLE_ADMIN')" >
               <a th:href="@{/book/list}" target="iframe">
                  <i class="fa fa-circle-o"></i>图书阅读
               </a>
            </li>
            <li sec:authorize="hasAuthority('ROLE_ADMIN')">
               <a th:href="@{/book/admin/manag}" target="iframe">
                  <i class="fa fa-circle-o"></i>图书管理
               </a>
```

```
 19              </li>
 20          </ul>
 21       </section>
 22       <!-- /.sidebar -->
 23    </aside>
 24    <!-- 导航侧栏 /-->
 25    <!-- 内容展示区域 -->
 26    <div class="content-wrapper">
 27       <iframe         width="100%"         id="iframe"         name="iframe"
onload="SetIFrameHeight()"
 28              frameborder="0" "></iframe>
 29    </div>
 30 </div>
 31 </body>
 32 </html>
```

在上述代码中，第 2 行代码引入 Spring Security 安全标签的名称空间，并指定安全标签的前缀为 sec；第 10 行代码使用 sec 标签的 authorize 属性指定当前认证的用户角色为 ROLE_COMMON 或者 ROLE_ADMIN 时，才在页面中显示该<li>标签中的内容；第 15 行代码使用 sec 标签的 authorize 属性指定当前认证的用户角色为 ROLE_ADMIN 时，才在页面中显示该<li>标签中的内容。

3. 效果测试

重启 chapter07 项目进行效果测试，项目启动后，使用用户 zhangsan 的信息进行登录，登录成功后，后台首页效果如图 7-23 所示。

从图 7-23 可以看出，用户 zhangsan 登录成功后，后台首页左侧的菜单只显示“图书阅读”，说明成功为角色是 ROLE_COMMON 的用户显示了对应的菜单。

使用用户 lisi 的信息进行登录，登录成功后，后台首页效果如图 7-24 所示。

图7-23 后台首页效果（1）

图7-24 后台首页效果（2）

从图 7-24 可以看出，用户 lisi 登录成功后，后台首页左侧的菜单显示了“图书阅读”和“图书管理”两个菜单，说明不同权限的用户登录后，后台首页成功显示了用户角色对应的内容，在 Thymeleaf 页面使用 Spring Security 安全标签可成功根据用户角色动态展示菜单。

7.5 Spring Security 会话管理和用户退出

用户认证通过后，为了避免用户的每次操作都进行认证，可以将用户信息保存在会话中。会话就是系统为了保持当前用户的登录状态所提供的机制，常见的有基于 Session 方式、基于 Token 方式等。Spring Security 提供会话管理功能，只需要配置即可使用。同时，如果想结束当前会话，可以在自定义退出功能中销毁会话中的用户信息。下面将在 Spring Boot 项目中基于 Spring Security 实现会话管理和用户退出。

7.5.1　会话管理

Spring Security 的会话管理是其核心功能之一，它允许开发者控制和定制用户会话的行为。下面分别从获取用户信息和会话控制这两方面对 Spring Security 的会话管理进行讲解。

1. 获取用户信息

Spring Security 对用户信息认证通过后，会将用户信息存入 Spring Security 应用的上下文对象 SecurityContext 中，SecurityContext 与当前线程进行绑定，需要获取用户信息时，可以通过 SecurityContextHolder 获取 SecurityContext 对象，进而使用 SecurityContext 对象获取用户信息，示例代码如下。

```
1  @RestController
2  public class LoginController {
3      @RequestMapping(value = "/getUsername")
4      public String getUsername() {
5          Authentication authentication =
6              SecurityContextHolder.getContext().getAuthentication();
7          if (!authentication.isAuthenticated()) {
8              return null;
9          }
10         UserDetails userDetails = (UserDetails) authentication.getPrincipal();
11         String username =userDetails.getUsername();
12         return username;
13     }
14 }
```

在上述代码中，第 5～6 行代码使用 SecurityContextHolder 类先获取了应用上下文对象 SecurityContext，进而通过 SecurityContext 相关方法获取了当前登录用户对应的认证信息；第 10 行代码通过 getPrincipal()方法获取了用户身份信息，在已认证的情况下获取到的是 UserDetails，在未认证的情况下获取到的是用户名。

2. 会话控制

默认情况下，Spring Security 会为每个登录成功的用户新建一个 Session 对象进行会话管理，开发者也可以根据具体的需求对 Session 的创建进行控制。Spring Security 管理 Session 的创建策略有以下四种。

- always：如果没有 Session 就创建一个。
- ifRequired：如果需要就创建一个 Session，是默认的创建策略。
- never：Spring Security 将不会创建 Session，但是如果项目中其他地方创建了 Session，那么 Spring Security 可以使用它。
- stateless：Spring Security 将绝对不会创建 Session，也不使用 Session。

当项目中不想自动创建 Session，但是想要使用项目中其他地方创建的 Session 时，可以选择使用 never 策略，示例代码如下。

```
@Bean
public  SecurityFilterChain securityFilterChain(HttpSecurity http) throws
Exception {
        http.sessionManagement()
                .sessionCreationPolicy(SessionCreationPolicy.NEVER);
    return http.build();
```

```
}
```

如果项目中允许创建 Session，默认情况下 Session 的超时时间为 30 分钟，如果想要对 Session 默认的超时时间进行修改，可以在 Spring Boot 的配置文件中设置 Session 的超时时间，示例代码如下。

```
server:
  servlet:
    session:
      timeout: 86400s
```

在上述代码中，设置 Session 的超时时间为 86400 秒。

7.5.2 用户退出

Spring Security 默认实现了用户退出的功能，用户退出主要考虑退出后会话如何管理以及跳转到哪个页面。HttpSecurity 类提供了 logout()方法开启退出登录的支持，默认触发用户退出操作的 URL 为“/logout”，用户退出时同时也会清除 Session 等默认用户配置。

用户退出登录的逻辑是由过滤器 LogoutFilter 执行的，但是项目开发时一般不会选择直接操作 LogoutFilter，而是通过 LogoutConfigurer 对 LogoutFilter 进行配置，HttpSecurity 类 logout()方法的返回值就是一个 LogoutConfigurer 对象，该对象提供了一系列设置用户退出的方法，其中主要的方法如表 7-4 所示。

表 7-4 用户退出的主要方法

方法	作用
logoutUrl(String logoutUrl)	用户退出处理控制 URL，默认为 post 请求的/logout
logoutSuccessUrl(String logoutSuccessUrl)	用户退出成功后的重定向地址
logoutSuccessHandler(LogoutSuccessHandler logoutSuccessHandler)	用户退出成功后的处理器设置
deleteCookies(String... cookieNamesToClear)	用户退出后删除指定 Cookie
invalidateHttpSession(boolean invalidateHttpSession)	用户退出后是否立即清除 Session，默认为 true
clearAuthentication(boolean clearAuthentication)	用户退出后是否立即清除 Authentication 用户认证信息，默认为 true

在了解了 Spring Security 中关于用户退出的相关方法后，下面通过案例演示用户退出的实现。

1. 设置用户退出的请求路径

要实现自定义用户退出功能，必须先在某个页面定义用户退出链接或者按钮。项目后台首页 main.html 右上方有一个“注销”菜单，可以在该菜单中设置用户退出的请求路径，页面的核心代码如文件 7-20 所示。

文件 7-20 main.html

```
<html xmlns:th="http://www.thymeleaf.org"
      xmlns:sec="http://www.thymeleaf.org/extras/spring-security">
<html>
  <body class="hold-transition skin-green sidebar-mini">
    <!-- ……其他代码 -->
      <!-- 头部导航 -->
```

```
7         <nav class="navbar navbar-static-top">
8            <div class="navbar-custom-menu">
9               <ul class="nav navbar-nav">
10                    <!-- ……其他代码 -->
11                 <li class="dropdown user user-menu">
12                    <a th:href="@{/logout}">
13                       <span class="hidden-xs">注销</span>
14                    </a>
15                 </li>
16              </ul>
17           </div>
18        </nav>
19     </header>
20     <!-- ……导航侧栏 -->
21     <!-- ……内容展示区域 -->
22 </div>
23 </body>
24 </html>
```

在上述代码中，第 12～14 行代码为“注销”设置了超链接，单击该超链接时会向项目路径“/logout”发送请求。需要说明的是，Spring Boot 项目中引入 Spring Security 框架后会自动开启 CSRF 防护功能，如果没有关闭 CSRF 防护功能，用户退出的请求必须使用 POST 请求。

2. 效果测试

重启 chapter07 项目进行效果测试，项目启动成功后，使用正确的用户信息登录后进入后台首页，单击后台首页页面右上角的“注销”，效果如图 7-25 所示。

图7-25　注销用户登录

从图 7-25 可以看出，页面跳转到登录页面。此时，在浏览器中发起与登录无关的其他请求，都会跳转回登录页面。说明单击“注销”超链接后，向项目的“/logout”路径发送请求，Spring Security 会根据默认的用户退出逻辑处理该请求，用户退出过程中清除了 Session 以及之前认证过的用户信息，实现了用户退出。

7.6　本章小结

本章主要对 Spring Boot 项目中的安全管理进行了讲解。首先讲解了安全框架概述；然

后讲解了 Spring Security 基础入门；接着讲解了 Spring Security 认证管理和授权管理；最后对 Spring Security 会话管理和用户退出进行了讲解。通过本章的学习，希望读者可以在 Spring Boot 项目中正确使用 Spring Security 进行安全管理。

7.7 本章习题

一、填空题

1．Spring Security 默认提供的用户名是_____。

2．_____中存储了当前用户的认证和权限信息。

3．Spring Security 5 如果不想对输入的密码进行加密，需要在密码前使用_____进行标注。

4 .Spring Security 中基于 HttpSecurity 的_____方法可以配置 CSRF 跨站请求伪造防护功能。

5．Spring Security 中默认的 Session 创建策略是_____。

二、判断题

1．授权是指验证用户是否有权限访问特定资源的过程。（　　）

2．Spring Security 安全管理的实现主要是由过滤器链中一系列过滤器相互配合完成。（　　）

3 .AuthenticationManager 接口是认证相关的核心接口，也是发起认证的出发点。（　　）

4．Spring Security 自定义身份认证后，项目启动时不会在控制台中输出 Spring Security 生成的随机密码。（　　）

5．使用 Spring Security 时，受保护的 Web 资源将被 SecurityFilterChain 中 FilterSecurity Interceptor 的实例对象拦截。（　　）

三、选择题

1．下列选项中，关于 Spring Security 的作用描述正确的是（　　）。

A．集成其他 Spring 框架。

B．提供用户认证和授权功能。

C．管理 Spring 应用程序的依赖关系。

D．管理开源库的版本控制。

2．下列选项中，Spring Security 处理所有身份验证请求的类是（　　）。

A．AuthenticationManager　　B．AuthenticationProvider

C．UserDetails　　D．AccessDecisionManager

3．下列选项中，Spring Security 默认使用的加密算法是（　　）。

A．BCryptPasswordEncoder

B．Pbkdf2PasswordEncoder

C．SCryptPasswordEncoder

D．DelegatingPasswordEncoder

4．下列选项中，开启 Spring Security 基于方法的安全认证机制的注解是（　　）。

A．@Secured　　B．@RolesAllowed

C．@PreAuthorize　　D．@EnableGlobalMethodSecurity

5．下列选项中，Spring Security 管理 Session 时如果没有 Session 就创建一个的策略是（　　）。

A．always　　B．ifRequired

C．never　　D．stateless

第8章

Spring Boot消息服务

学习目标

★ 熟悉常见消息中间件，能够说出4种常见的消息中间件名称和特点

★ 熟悉使用消息服务的好处，能够通过不同的场景说出使用消息服务的好处

★ 了解RabbitMQ简介，能够说出RabbitMQ消息代理流程

★ 熟悉RabbitMQ工作模式，能够说出RabbitMQ支持的工作模式，以及每种模式的工作原理

★ 掌握RabbitMQ的下载、安装和配置，能够在Windows平台安装和配置RabbitMQ，并成功启动RabbitMQ

★ 掌握RabbitMQ入门案例，能够独立实现RabbitMQ入门案例

★ 掌握Spring Boot整合RabbitMQ环境搭建，能够在Spring Boot项目中整合RabbitMQ

★ 掌握使用Publish/Subscribe模式实现消息服务，能够在Spring Boot项目中使用Publish/Subscribe模式实现消息服务

★ 掌握使用Routing模式实现消息服务，能够在Spring Boot项目中使用Routing模式实现消息服务

★ 掌握使用Topics模式实现消息服务，能够在Spring Boot项目中使用Topics模式实现消息服务

在实际项目开发中，有时候需要与其他系统进行集成完成相关业务功能，这种情况最原始的做法是程序内部相互调用，除此之外，还可以使用消息服务中间件进行业务处理，使用消息服务中间件处理业务能够提升系统的异步通信和扩展解耦能力。Spring Boot为消息服务管理提供了非常好的支持，下面将对Spring Boot消息服务的原理和整合使用进行详细讲解。

8.1 消息服务概述

Java中的消息服务是指不同应用程序之间或同一个应用程序的不同组件之间通信的API，包括创建、发送、读取消息等，用于支持Java应用程序开发。在J2EE中，当两个组件使用Java消息服务进行通信时，发送消息的组件通常称为消息生产者，使用发送过来的数据的组件称为消息消费者，消息生产者与消息消费者之间并不是直接相连的，而是通过一个共同的消息收发服务连接起来，消息生产者和消息消费者双方无须相互了解消息服务的实

现细节，只需了解交换消息的格式即可。消息服务的这种机制实现了组件之间的解耦，因此更加灵活。下面对常用消息中间件以及使用消息服务的优势进行讲解。

8.1.1　常用消息中间件

消息队列（Message Queue，MQ）是一种能实现消息生产者到消息消费者单向通信的通信模型，通常将实现了这个模型的组件称为消息队列中间件（简称消息中间件）。消息中间件通过高效可靠的消息传递机制进行与平台无关的数据交流，并基于数据通信来进行分布式系统的集成。目前，开源的消息中间件可谓是琳琅满目，能让大家耳熟能详的有很多，例如 ActiveMQ、RabbitMQ、Kafka、RocketMQ 等，也有直接使用 Redis 充当消息队列的案例，而这些消息中间件各有侧重，在实际选型时，需要结合具体需求来选择。下面对常用的消息中间件进行介绍。

1. ActiveMQ

ActiveMQ 是采用 Java 语言编写的完全基于 Java 消息服务（Java Message Service，JMS）规范的面向消息中间件，它为应用程序提供高效的、可扩展的、稳定的、安全的企业级消息通信。ActiveMQ 丰富的 API 和多种集群构建模式使其成为业界老牌的消息中间件，在中小型企业中应用广泛。相较于后续出现的 RabbitMQ、RocketMQ、Kafka 等消息中间件来说，ActiveMQ 性能相对较弱，在如今的高并发、大数据处理的场景下显得力不从心，经常会出现一些问题，例如消息延迟、堆积、堵塞等。

2. RabbitMQ

RabbitMQ 是使用 Erlang 语言开发的开源消息队列，支持高级消息队列协议（Advanced Message Queuing Protocol，AMQP）、可扩展通信和表示协议（Extensible Messaging and Presence Protocol，XMPP）等。AMQP 是为应对大规模并发活动提供统一消息服务的应用层标准高级消息队列协议，专为面向消息的中间件设计，该协议多用在企业系统内对数据一致性、稳定性和可靠性要求很高的场景，对性能和吞吐量的要求不是很高。RabbitMQ 具有并发能力强、性能好、消息延迟低、社区活跃、管理界面丰富等特性，使其在应用开发中越来越受欢迎。

3. Kafka

Kafka 是一种采用 Scala 和 Java 语言编写的高吞吐量分布式发布订阅消息系统，提供了快速的、可扩展的、分布式的、分区的和可复制的日志订阅服务，其主要特点是追求高吞吐量，适用于会产生大量数据的互联网服务的数据收集业务。

4. RocketMQ

RocketMQ 使用纯 Java 语言编写，具有高吞吐量、高可用性、适合大规模分布式系统应用的特点。RocketMQ 的思路起源于 Kafka，对消息的可靠传输和事务性处理做了优化，目前被广泛应用于交易、充值、流计算、消息推送、日志流式处理场景。

在实际项目技术选型时，在没有特别要求的场景下，通常会选择使用 RabbitMQ 作为消息中间件，本章也主要基于 RabbitMQ 进行讲解。如果针对的是大数据业务，推荐使用 Kafka 或者 RocketMQ 作为消息中间件。

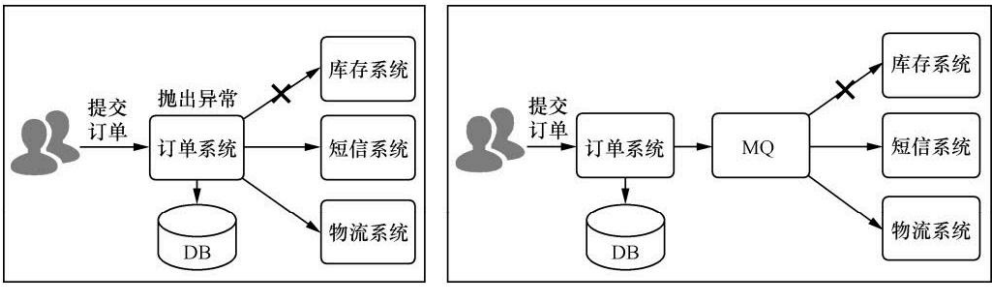
8.1.2　使用消息服务的好处

在多数应用尤其是分布式系统中，消息服务是不可或缺的重要部分，它使用起来比较

简单，同时解决了不少难题，例如应用解耦、异步提速、流量削峰、分布式事务管理等，使用消息服务可以实现一个高性能、高可用性、高扩展性的系统。下面使用实际开发中的若干场景来分析和说明使用消息服务的好处。

1. 应用解耦

场景说明：用户下单后，会调用系统的订单系统、库存系统、短信系统、物流系统，期间任意一个系统都存在出现异常的可能。下面使用图示方式直观地展示上述需求的不同处理方式，如图 8-1 所示。



（1）使用传统方式处理　　（2）使用消息服务模式

图8-1　应用解耦场景说明图示

在图 8-1 中，如果使用传统方式处理订单业务，用户提交订单后，订单系统保存订单信息后会依次调用库存系统、短信系统、物流系统，这种方式有一个很大的问题是：一旦其中任意一个系统出现异常，订单服务会失败，从而导致订单提交失败，核心业务的订单系统因为其他业务系统出现异常而导致订单提交失败，这非常不合理。

如果使用消息服务模式，保存用户提交的订单信息后，会快速将订单信息写入消息队列，其他服务会监听并读取到消息队列中的订单，即使单个系统发生异常也不会影响其他系统，并且出现异常的系统修复后还可以继续从消息队列中重新读取订单信息，重新执行当前系统的业务。相较于传统方式，消息服务模式只需确保消息队列的高可用性，服务之间的耦合性更低，容错性和可维护性更高。

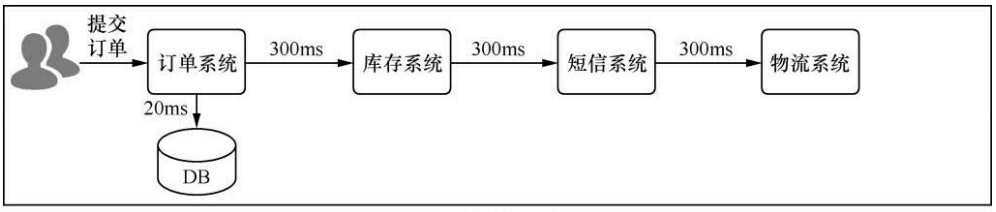
2. 异步提速

场景说明：用户提交订单后，系统需要将订单信息写入数据库，调用库存系统扣减商品库存，调用短信系统给用户发送短信，调用物流系统通知物流部门配送订单。下面使用图示的方式直观展示上述场景的不同处理方式，如图 8-2 所示。

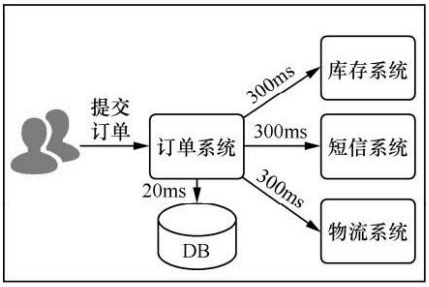
在图 8-2 中，针对上述提交订单业务的场景需求，处理方式有三种，具体说明如下。

- 串行处理方式：用户提交订单后，服务器会先将订单信息写入数据库，并依次调用库存系统、短信系统、物流系统，服务器只有在调用所有系统完毕后才会将处理结果返回客户端。这种串行处理消息的方式耗时为 920ms（20ms+300ms+300ms+300ms），响应慢，用户体验不友好。
- 并行处理方式：用户提交订单后，将订单信息写入数据库，同时调用库存系统、短信系统、物流系统，最后返回给客户端，这种并行处理的方式耗时为 320ms（20ms+300ms），在一定程度上提高了后台业务处理的效率，但如果遇到较为耗时的业务处理，仍然显得不够完善。
- 消息服务处理方式：可以在业务中嵌入消息服务进行业务处理，这种方式先将订单

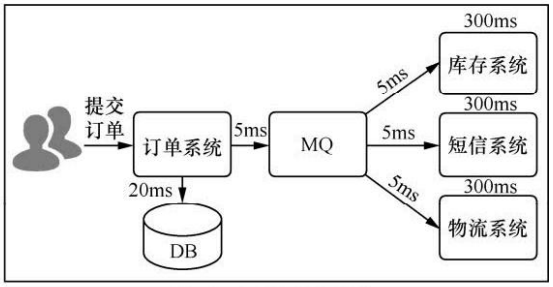
信息写入数据库，在极短的时间内将订单信息写入消息队列后即可返回响应信息，此时前端业务只需知道订单提交的结果，而库存系统、短信系统、物流系统的业务会自动读取消息队列中的相关消息进行后续业务处理，耗时为 25ms（20ms+5ms），大幅降低了响应所需的时间，提升了用户体验和系统吞吐量。



（1）串行处理



（2）并行处理

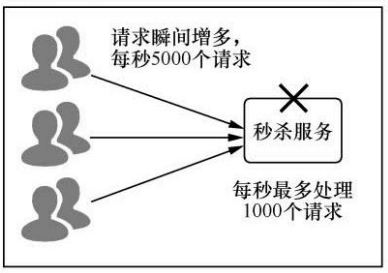


（3）使用消息服务处理

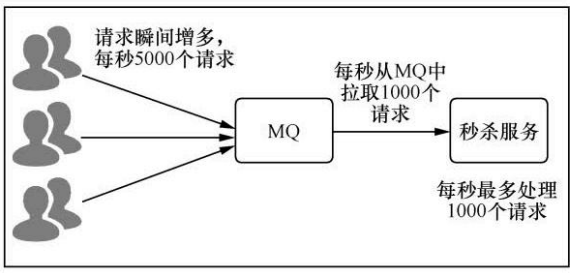
图8-2　异步处理场景说明图示

3. 流量削峰

场景说明：秒杀活动是流量削峰的一种应用场景，由于服务器处理资源能力有限，因此出现峰值时很容易造成服务器宕机、用户无法访问的情况。为了解决这个问题，通常会采用消息队列缓冲瞬时高峰流量，对请求进行分层过滤，从而过滤掉一些请求。图 8-3 描述的是流量削峰场景的处理方式。



（1）使用传统处理



（2）使用消息服务

图8-3　流量削峰场景说明图示

针对上述秒杀业务的场景需求，如果专门增设服务器来应对秒杀活动期间的请求瞬时高峰的话，在非秒杀活动期间，这些多余的服务器和配置显得有些浪费；如果不进行有效处理的话，秒杀活动瞬时高流量请求有可能压垮服务器。因此，在秒杀活动中加入消息服务是较为理想的解决方案。

通过在应用中加入消息服务，当瞬时高并发请求到来时，服务器接收所有请求，并将它们写入消息队列中。当消息队列中的请求数量达到最大值时，用户请求将被转发到错误

提示页面，这样可控制参加活动的人数。当消息队列缓存所有用户请求之后，秒杀服务再根据秒杀规则对消息逐一进行处理，通过这种方式缓解了瞬时高流量对秒杀业务处理系统的压力，提高了系统的稳定性。

4. 分布式事务管理

场景说明：在分布式系统中，分布式事务是开发中必须要面对的技术难题，怎样保证分布式系统的请求业务处理的数据一致性通常是需要重点考虑的问题。针对这种分布式事务管理的情况，目前较为可靠的处理方式是基于消息队列的二次提交，在失败的情况下可以进行多次尝试，或者基于队列数据进行回滚操作，因此，在分布式系统中加入消息服务是一个既能保证性能不变，又能保证业务一致性的方案。

针对这种分布式事务处理的需求，以图示的方式展示使用消息服务的处理机制，如图 8-4 所示。

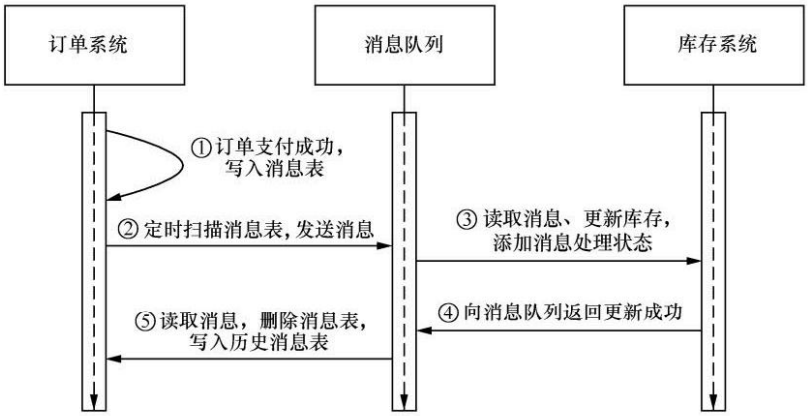


图8-4 分布式事务管理场景说明图示

针对上述分布式事务管理的场景需求，如果使用传统方式在订单系统中写入订单支付成功的信息后，再远程调用库存系统进行库存更新，一旦库存系统异常，很有可能导致库存更新失败而订单支付成功的情况，从而导致数据不一致。针对这种分布式系统的事务管理，通常会在分布式系统之间加入消息服务来进行管理。

在图 8-4 中，订单支付成功后，写入消息表；然后定时扫描消息表，将消息写入消息队列中，库存系统会立即读取消息队列中的消息进行库存更新，同时添加消息处理状态；接着，库存系统向消息队列中写入库存处理结果，订单系统会立即读取消息队列中的库存处理状态。如果库存服务处理失败，订单服务还会重复扫描并发送消息表中的消息，让库存系统进行最终一致性的库存更新；如果处理成功，订单服务直接删除消息表数据，并写入到历史消息表。

8.2 RabbitMQ 快速入门

8.2.1 RabbitMQ 简介

AMQP 是一种与平台无关的线路级的消息中间件协议，其通过制定协议来统一数据交互的格式。AMQP 是跨语言的，它只制定协议，不规定实现语言和实现方式。RabbitMQ 是基于 AMQP 标准，使用 Erlang 语言开发的消息中间件，在 Spirng Boot 中对 RabbitMQ 进行了集成管理。

在所有的消息服务中，消息中间件都会作为一个第三方消息代理，接收消息生成者发

布的消息，并推送给消息消费者。不同消息中间件内部转换消息的细节不同，图 8-5 展示的是 RabbitMQ 的消息代理流程。

从图 8-5 可以看出，RabbitMQ 的消息代理流程中有很多内部组件，下面对内部主要的组件进行说明，具体如下。

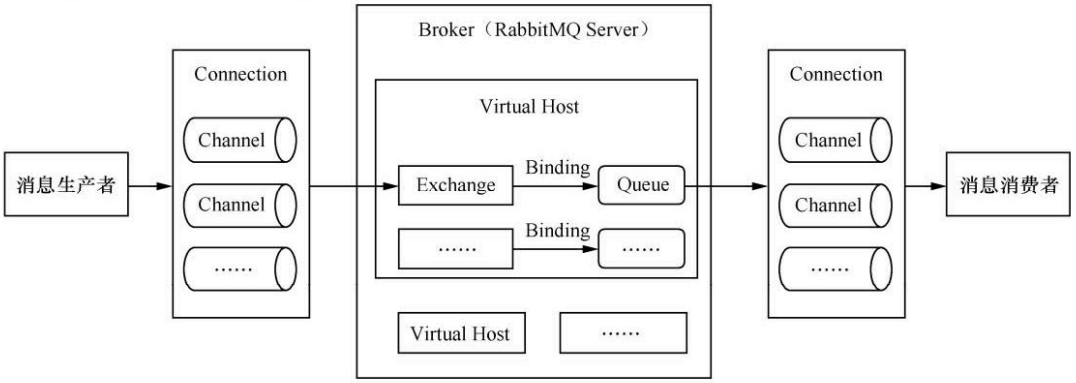


图8-5　RabbitMQ的消息代理流程

- Connection：代表客户端（包括消息生产者和消息消费者）与 RabbitMQ 之间的连接。
- Channel：消息通道，位于连接内部，负责实际的通信。在客户端的每个连接里，可建立多个 Channel。
- Broker：表示消息队列服务器实体，Broker 会维护从消息生产者到消息消费者的路线，保证数据能按照指定的方式进行传输。
- Virtual Host：虚拟主机，包含交换机、消息队列等对象。一个 Broker 里可以有多个 Virtual Host。当多个不同的用户使用同一个 RabbitMQ Server 提供的服务时，可以划分出多个 Virtual Host，每个用户在自己的 Virtual Host 创建对应的交换机、消息队列。
- Exchange：交换机，用于接收消息生产者发送的消息，并根据分发规则将这些消息路由给服务器中的队列。
- Queue：队列，是消息的载体，每个消息都会被投到一个或多个队列中，等待消息消费者连接到这个队列将其取走。
- Binding：交换机和消息队列之间的虚拟连接。

学习完 RabbitMQ 内部主要组件的相关内容后，根据图 8-5 总结了 RabbitMQ 消息代理流程，具体如下。

消息生产者（发送消息的程序）与消息队列服务器实体建立连接后，向指定的虚拟主机发送消息。

虚拟主机内部的交换机接收消息生产者发送过来的消息，将消息传递并存储到与之绑定的消息队列中。

消息消费者（等待接收消息的程序）通过网络连接与消息队列服务器实体建立连接，在连接内部使用消息通道进行消息的最终消费。

在使用 RabbitMQ 时，应增强信息安全意识，在传递重要的信息时，应注意信息的加密传输，确保信息不被泄漏，从而有效地保证信息安全，为维护健康的网络环境贡献自己的一份力量。

8.2.2 RabbitMQ 工作模式

RabbitMQ 消息中间件针对不同的服务需求，提供了多种工作模式，下面对 RabbitMQ 支持的工作模式和工作原理进行简要说明。

1. 简单模式

RabbitMQ 是一个消息代理，它接收和转发消息。RabbitMQ 的简单模式不用声明交换机，只需要定义一个队列，消息生产者会将消息交给默认的交换机，默认的交换机将获取到的信息绑定到消息生产者对应的队列，消息消费者监听这个队列，当队列中有消息时，消息消费者执行消息消费。简单模式的流程示意图如图 8-6 所示。

图 8-6 中 P 表示消息生产者，C 表示消息消费者，P 和 C 中间连续的矩形图案表示队列。消息被消息消费者从队列中取出后，消息默认会自动从队列中删除。

2. Work Queues

Work Queues 模式即工作队列模式，在 Work Queues 模式中，同样不需要设置交换机，RabbitMQ 会使用内部默认交换机进行消息转换，只需要指定唯一的消息队列进行消息传递。Work Queues 模式和简单模式的主要区别在于，Work Queues 模式可以有多个消息消费者。在这种模式下，多个消息消费者通过轮询的方式依次接收消息队列中存储的消息，一旦消息被某一个消息消费者接收，消息队列会将消息移除，而接收并处理消息的消息消费者必须在消费完一条消息后再准备接收下一条消息。Work Queues 模式的流程示意图如图 8-7 所示。

图8-6 简单模式的流程示意图

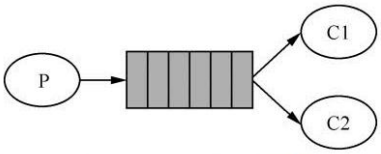


图8-7 Work Queues模式的流程示意图

从上述分析可以发现 ,Work Queues 模式适用于那些较为繁重且可以进行拆分处理的业务，这种情况下可以将业务分派给多个消息消费者来轮流进行处理。

3. Publish/Subscribe

Publish/Subscribe 模式即发布/订阅模式，在 Publish/Subscribe 模式中，必须先手动配置一个 fanout 类型的交换机，消息生产者将消息发送到交换机，同时会将消息路由到每一个消息队列上，然后每个消息队列都可以对相同的消息进行接收和存储，进而由各自消息队列关联的消息消费者进行消费。Publish/Subscribe 模式的流程示意图如图 8-8 所示。

从上述分析可以发现，Publish/Subscribe 工作模式适用于进行相同业务功能处理的场合。例如，用户注册成功后，需要同时发送邮件通知和短信通知，那么邮件服务消费者和短信服务消费者需要共同消费“ 用户注册成功 ”这一条消息。

4. Routing

Routing 模式即路由模式，在 Routing 模式中，必须先配置一个 direct 类型的交换机，并指定一个 Routing Key。队列与交换机的绑定也需要指定一个 Routing Key，发送消息时交换机不再把消息交给每一个绑定的队列，而是根据消息的 Routing Key 进行判断，只有队列的 Routing Key 与消息的 Routing Key 完全一致的，才会接收到消息，最后由消息消费者进行消费。Routing 模式的流程示意图如图 8-9 所示。

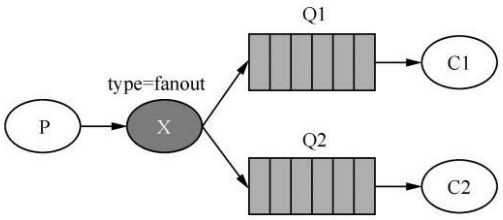


图8-8　Publish/Subscribe模式的流程示意图

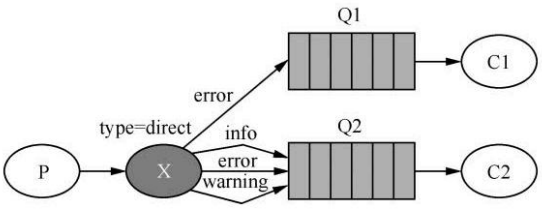


图8-9　Routing模式的流程示意图

从上述分析可以发现，Routing 工作模式适用于对不同类型消息进行分类处理的场合。例如日志收集处理，用户可以配置不同的路由键值分别对不同级别的日志信息进行分类处理。

5. Topics（通配符模式）

Topics 模式即通配符模式，在 Topics 模式中必须先配置一个 topic 类型的交换机，并指定不同的 Routing Key，根据 Routing Key 将对应的消息从交换机路由到不同的消息队列进行存储，然后由消息消费者进行消费。Topics 模式与 Routing 模式的主要不同在于，Topics 模式设置的路由键包含通配符，其中，“ # ”用于匹配多个字符，“ * ”用于匹配一个字符，然后与其他字符一起用“ . ”进行连接，从而组成动态路由键。在发送消息时可以根据需求设置不同的路由键，从而将消息路由到不同的消息队列。Topics 模式的流程示意图如图 8-10 所示。

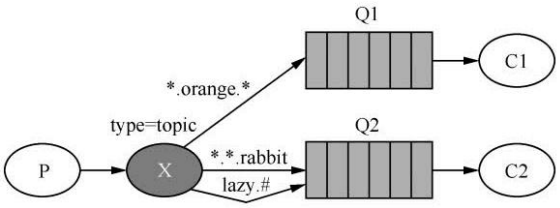


图8-10　Topics模式的流程示意图

从上述分析可以发现，Topics 工作模式适用于根据不同需求动态传递处理业务的场合。例如，一些订阅客户只喜欢接收邮件消息，一些订阅客户只喜欢接收短信消息，而有些订阅客户可以同时接收邮件消息和短信消息，那么可以根据客户需求进行动态路由匹配，从而将订阅消息分发到不同的消息队列中。

6. RPC

RPC 模式与 Work Queues 模式主体流程相似，都不需要设置交换机，只需要指定唯一的消息队列进行消息传递。RPC 模式与 Work Queues 模式的主要不同在于，RPC 模式是一个回环结构，主要针对分布式架构的消息传递业务，客户端（Client）先发送消息到消息队列，远程服务器端（Server）获取消息，然后再写入另一个消息队列，向原始客户端（Client）响应消息处理结果。RPC 模式的流程示意图如图 8-11 所示。

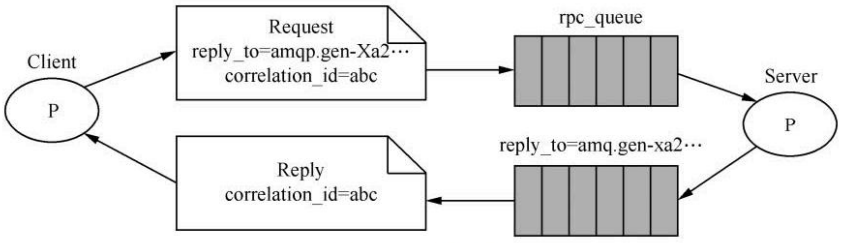


图8-11　RPC模式的流程示意图

从上述分析可以发现，RPC 工作模式适用于远程服务调用的业务处理场合。例如，在分布式架构中必须考虑的分布式事务管理问题。

7. Publisher Confirms

Publisher Confirms 模式即发布者确认模式，是 AMQP 0.9.1 协议的 RabbitMQ 扩展，默认情况下不启用，发布者在通道上启用发布者确认后，代理将异步确认客户端发布的消息。Publisher Confirms 模式最大的好处在于它是异步的，一旦发布一条消息，消息生产者就可以在等待信道返回确认的同时继续发送下一条消息，当消息最终得到确认之后，消息生产者便可以通过回调方法来处理该确认消息。

上述主要对 RabbitMQ 支持的 7 种工作模式及原理进行了说明，其中有些工作模式还可以嵌套使用，例如在发布订阅模式中加入工作队列模式。这里介绍的 7 种工作模式中，Publish/Subscribe、Routing、Topics 和 RPC 模式在开发中是较为常用的几种工作模式。

8.2.3 RabbitMQ 的下载、安装和配置

在使用 RabbitMQ 之前必须预先安装，RabbitMQ 提供了多平台的安装包，例如 Linux、Windows、macOS、Docker 等。为了方便读者操作，这里选择在 Windows 平台安装 RabbitMQ。

1. RabbitMQ 下载

在 RabbitMQ 官网首页的 Get Started 版块中单击“ Download+Installation ”按钮进入 RabbitMQ 的下载页面，如图 8-12 所示。

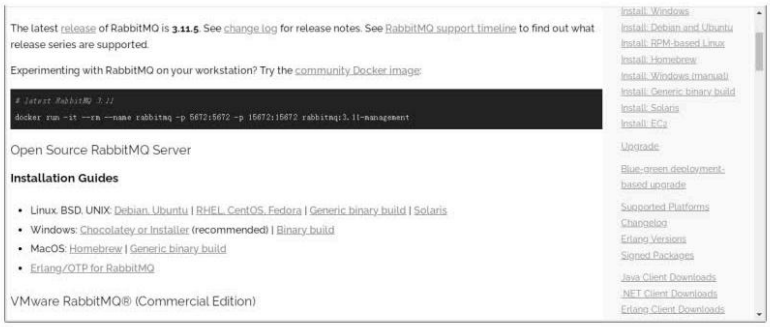


图8-12 RabbitMQ的下载页面

在图 8-12 中可以看到多个 RabbitMQ 支持的系统平台，Windows 平台提供了 Chocolatey or Installer 和 Binary build 两种安装方式，其中 recommended（推荐）的为 Chocolatey or Installer 方式。单击图 8-12 中的“ Chocolatey or Installer ”超链接，进入基于 Windows 平台的安装包下载页面，如图 8-13 所示。

图8-13 基于Windows平台的安装包下载页面

从图 8-13 可以看到，截至本书编写时 RabbitMQ 提供的最新版本为 rabbitmq-server-3.11.5.exe，单击“ rabbitmq-server-3.11.5.exe ”下载该版本的安装包。本书对应的资源中也

提供了对应的安装包，读者也可以选择直接从资源中获取该安装包。

2. RabbitMQ 安装和配置

RabbitMQ 是使用 Erlang 语言开发的，安装 RabbitMQ 之前需要先安装 Erlang 语言包。RabbitMQ 3.11.5 需要 Erlang 的最小版本为 25.0、最大版本为 25.1。读者可以自行到 Erlang 官网下载对应的安装包，也可以直接使用本书对应资源中提供的安装包。

为了避免安装时因权限不够而导致安装不成功的情况，安装时以管理员身份运行安装包，并且不要安装在中文或带空格的文件路径下。RabbitMQ 安装包和 Erlang 语言包的安装都非常简单，只需要双击下载的安装文件，然后单击“ 下一步 ”按钮、“ 安装 ”按钮即可完成，在此不对安装过程进行讲解。

完成 Erlang 和 RabbitMQ 的安装后 ,进一步对 RabbitMQ 的启动进行配置 ,具体步骤如下。

（1）配置环境变量

在 Windows 环境下首次执行 RabbitMQ 的安装后，环境变量中会自动增加一个变量名为 ERLANG_HOME 的系统变量，ERLANG_HOME 的值是 Erlang 安装的具体路径，无须手动修改。只需要在 Path 系统变量中添加值“ %ERLANG_HOME%\bin; ”就可以将 Erlang 的 bin 路径添加到 Path 系统变量中。

为了确保 Erlang 安装成功，以及环境变量设置成功，在 cmd 窗口中任意路径下输入 erl 命令进行验证，如图 8-14 所示。

从图 8-14 可以看到 Erlang 的版本号，说明 Erlang 安装成功，并且环境变量设置成功。

（2）启动插件

编辑完环境变量后，在 cmd 窗口中进入 RabbitMQ 的 sbin 目录，并执行如下命令。

```
rabbitmq-plugins enable rabbitmq_management
```

上述命令为启动 RabbitMQ 的插件，执行该命令后的效果如图 8-15 所示。

图8-14　RabbitMQ可视化登录页面

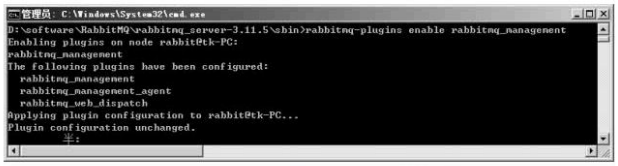


图8-15　启动RabbitMQ的插件

（3）启动 RabbitMQ

进入 RabbitMQ 的 sbin 目录，双击“ rabbitmq-server.bat ”文件启动 RabbitMQ 服务，此时会弹出 cmd 窗口显示启动情况，如图 8-16 所示。

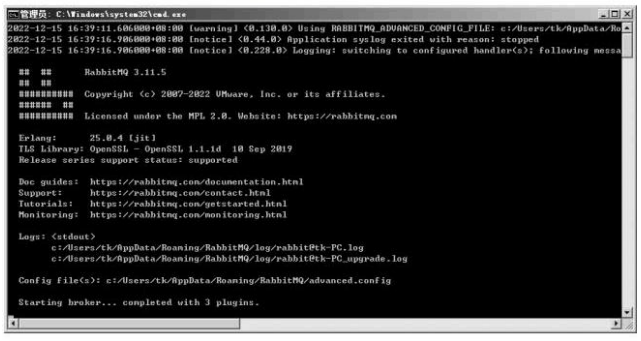


图8-16　启动RabbitMQ服务

RabbitMQ 默认提供了两个端口号 5672 和 15672，其中 5672 用作服务端口号，15672 用作可视化管理端口号。在浏览器中访问“ http://localhost:15672 ”，通过可视化的方式查看 RabbitMQ，效果如图 8-17 所示。

从图 8-17 可以看出，成功打开了 RabbitMQ 可视化页面，说明 RabbitMQ 服务启动成功。首次登录 RabbitMQ 可视化管理页面时需要进行用户登录，RabbitMQ 安装过程中默认提供了用户名和密码均为 guest 的用户，可以使用该账户进行登录。登录成功后会进入 RabbitMQ 可视化管理页面的首页，效果如图 8-18 所示。

如图 8-18 所示，在 RabbitMQ 可视化管理页面中显示有 RabbitMQ 相关的版本信息、用户信息等，同时提供了包括 Connections、Channels、Exchanges、Queues、Admin 在内的管理面板。

至此，RabbitMQ 的下载、安装和配置已经完成。

图8-17 RabbitMQ可视化页面

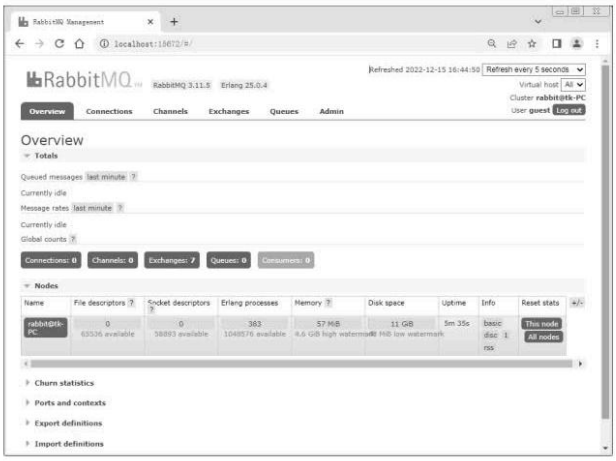


图8-18 RabbitMQ可视化管理页面首页

8.2.4 RabbitMQ 入门案例

在 RabbitMQ 的使用过程中，按需求实现程序对应的功能时，需要秉承强烈的责任感与担当精神，对所编写的代码负责，在代码中体现出对他人权益的尊重和保护。在消息队列中注意权限的设置，控制不同用户对不同队列的访问权限，以避免恶意用户对队列的滥用，确保队列中的信息不被恶意篡改，充分认识到程序系统对社会和用户的影响。

RabbitMQ 中提供了 queueDeclare()方法，用于声明创建队列的信息，queueDeclare()方法的具体定义如下。

```
AMQP.Queue.DeclareOk queueDeclare(String queue, boolean durable, boolean exclusive,
boolean autoDelete, Map<String, Object> arguments) throws IOException;
```

在上述代码的参数中，queue 表示队列名称。durable 表示是否持久化，队列的声明默认是存放在内存中的，即便队列中有数据，RabbitMQ 重启时也会丢失队列，如果想重启之后队列还存在，就要使队列持久化，需要设置 durable 为 true。exclusive 表示是否独占，即是否只能有一个消息消费者监听这个队列，以及当 Connection 关闭时是否删除队列。Auto Delete 表示当最后一个消息消费者断开连接之后队列是否自动被删除。arguments 表示其他参数。

RabbitMQ 创建队列后，可以通过 basicPublish()方法向 RabbitMQ 发送消息，basicPublish()

方法的具体定义如下。

```
void basicPublish(String exchange, String routingKey, BasicProperties props, byte[]
body);
```

在上述代码的参数中，exchange 表示交换机名称，不指定交换机时，RabbitMQ 会使用默认的交换机；routingKey 表示路由键的名称，交换器根据消息携带的路由键来决定消息交给哪个队列；props 表示消息的配置属性；body 表示发送的消息数据。

下面通过一个简单的入门程序来演示 RabbitMQ 的简单使用，具体实现步骤如下。

1. 创建项目

在 IDEA 中创建一个名称为 rabbitmq-simple 的 Maven 项目，并在项目的 pom.xml 文件中导入 RabbitMQ 的客户端依赖和 Maven 的插件，具体如文件 8-1 所示。

文件 8-1　pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
        http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <groupId>com.itheima</groupId>
    <artifactId>rabbitmq-simple</artifactId>
    <version>1.0-SNAPSHOT</version>
    <dependencies>
        <!--rabbitmq Java 客户端-->
        <dependency>
            <groupId>com.rabbitmq</groupId>
            <artifactId>amqp-client</artifactId>
            <version>5.6.0</version>
        </dependency>
    </dependencies>
    <build>
        <plugins>
            <plugin>
                <groupId>org.apache.maven.plugins</groupId>
                <artifactId>maven-compiler-plugin</artifactId>
                <version>3.8.0</version>
                <configuration>
                    <source>1.11</source>
                    <target>1.11</target>
                </configuration>
            </plugin>
        </plugins>
    </build>
</project>
```

2. 实现消息生产

在项目的 src/main/java 目录下创建消息生产类 Producer_HelloWorld，在该类中的 main() 方法中编写生产消息的代码，具体如文件 8-2 所示。

文件 8-2　Producer_HelloWorld.java

```
import com.rabbitmq.client.Channel;
```

```
 2  import com.rabbitmq.client.Connection;
 3  import com.rabbitmq.client.ConnectionFactory;
 4  import java.io.IOException;
 5  import java.util.concurrent.TimeoutException;
 6  public class Producer_HelloWorld {
 7      public static void main(String[] args) throws IOException, TimeoutException
{
 8          //1.创建连接工厂
 9          ConnectionFactory factory = new ConnectionFactory();
10          //2. 设置参数
11          factory.setHost("localhost");//设置连接的主机地址
12          factory.setPort(5672); //端口
13          factory.setVirtualHost("/");//设置虚拟机
14          factory.setUsername("guest");//设置用户名
15          factory.setPassword("guest");//设置密码
16          //3. 创建连接 Connection
17          Connection connection = factory.newConnection();
18          //4. 创建 Channel
19          Channel channel = connection.createChannel();
20          //5. 创建队列 Queue
21          //如果没有一个名字叫 hello_world 的队列，则会创建该队列，如果有则不会创建
22          channel.queueDeclare("hello_world",true,false,false,null);
23          String body = "hello rabbitmq～～～";
24          //6. 发送消息
25          channel.basicPublish("","hello_world",null,body.getBytes());
26      }
27 }
```

本案例使用 RabbitMQ 的简单模式实现消息的转发，所以不需要声明交换机。在上述代码中，第 9 行代码创建了一个连接工厂，该连接工厂可以创建用于连接 RabbitMQ 的连接对象。第 11～15 行代码设置连接工厂的参数，包括连接的 RabbitMQ 服务器地址、端口号、连接的虚拟主机、用户名和密码，由于本章的 RabbitMQ 安装后并没有对其相关配置进行重新设置或者添加，所以只能使用默认的配置参数连接 RabbitMQ，即如果没有进行连接参数的设置，连接工厂会使用默认的参数。本案例中也可以省略第 11～15 行代码的参数设置，如果读者自行对 RabbitMQ 的配置进行过设置，需要使用自己当前的参数进行设置。第 17～19 行代码通过连接工厂对象创建连接对象，并通过连接对象创建消息通道对象。第 22 行代码使用 queueDeclare()方法声明创建队列的信息。第 23～25 行代码设置发送的消息数据，并向 RabbitMQ 发送消息。

3. 实现消息消费

在项目的 src/main/java 目录下创建消息消费类 Consumer_HelloWorld，在该类中的 main()方法中编写消费消息的代码，具体如文件 8-3 所示。

文件 8-3　Consumer_HelloWorld.java

```
 1  import com.rabbitmq.client.*;
 2  import java.io.IOException;
 3  import java.util.concurrent.TimeoutException;
 4  public class Consumer_HelloWorld {
 5      public static void main(String[] args) throws IOException, TimeoutException
{
```

```
 6         //1.创建连接工厂
 7         ConnectionFactory factory = new ConnectionFactory();
 8         //2. 设置参数
 9         factory.setHost("172.16.98.133");//ip  默认值 localhost
10         factory.setPort(5672); //端口  默认值 5672
11         factory.setVirtualHost("/");//虚拟机 默认值/
12         factory.setUsername("guest");//用户名
13         factory.setPassword("guest");//密码
14         //3. 创建连接 Connection
15         Connection connection = factory.newConnection();
16         //4. 创建 Channel
17         Channel channel = connection.createChannel();
18         //5. 创建队列 Queue
19         //如果没有一个名字叫 hello_world 的队列，则会创建该队列，如果有则不会创建
20         channel.queueDeclare("hello_world",true,false,false,null);
21         // 接收消息
22         Consumer consumer = new DefaultConsumer(channel){
23            @Override
24            public void handleDelivery(String consumerTag, Envelope envelope,
25               AMQP.BasicProperties properties, byte[] body) throws IOException
{
26               System.out.println("consumerTag: "+consumerTag);
27               System.out.println("Exchange: "+envelope.getExchange());
28               System.out.println("RoutingKey: "+envelope.getRoutingKey());
29               System.out.println("properties: "+properties);
30               System.out.println("body: "+new String(body));
31            }
32         };
33         channel.basicConsume("hello_world",true,consumer);
34      }
35 }
```

消息消费者消费消息时，需要先与 RabbitMQ 建立连接，然后通过消息通道获取指定队列中的消息。在上述代码中，第 9～20 行代码依次创建连接工厂对象、设置连接工厂对象的参数、通过连接工厂对象创建连接对象、通过连接对象创建消息通道对象、通过消息通道对象声明连接的队列信息，这些内容与消息生产者中的代码一致，在此就不再重新对其内容进行说明。

第 22～32 行代码创建了一个 Consumer 消费者对象，由于 Consumer 是一个接口，在此使用 RabbitMQ 客户端中提供的 Consumer 子类 DefaultConsumer 创建消费者对象。其中，handleDelivery()为 DefaultConsumer 的回调方法，当收到消息后，会自动执行该方法。handleDelivery()方法中的参数中，consumerTag 为消费者的标识；envelope 封装了消息发送的编号、交换机的名称、消息发送的路由键等信息；properties 为发送消息时的 props 参数；body 为消息生产者发送的消息内容。

第 33 行代码使用 basicConsume()方法将 Consumer 与消息队列绑定，basicConsume()方法的定义具体如下。

```
String basicConsume(String queue, boolean autoAck, Consumer callback) throws
IOException;
```

在上述代码的参数中，queue 表示 Consumer 绑定的队列名称；autoAck 表示是否自动确

认，如果为 true，表示 Consumer 接收到消息后，会自动发送确认消息给消息队列，消息队列会将这条消息从消息队列里删除；callback 就是 Consumer 对象，用于处理接收到的消息。

4. 测试效果

运行文件 8-2 生产消息，此时，登录 RabbitMQ 可视化页面，查看 Queues 面板中的内容，如图 8-19 所示。

从图 8-19 可以看到，Queues 面板中此时包含了一个名称为 hello_world 的队列，该队列中包含一条未读的消息，说明消息生产者成功生产了一条消息，并存储在 hello_world 队列中。

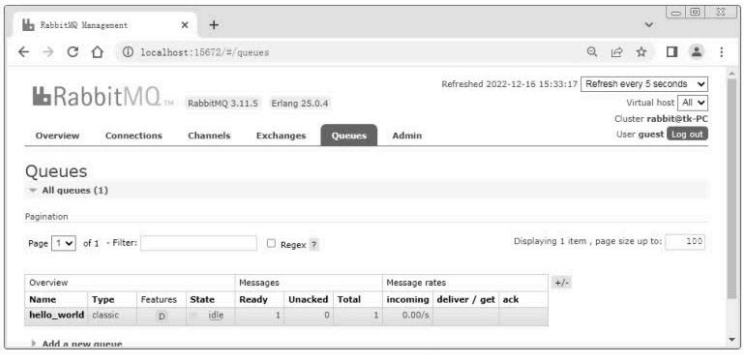


图8-19 Queues面板中的内容（1）

运行文件 8-3 消费消息，控制台输出内容如图 8-20 所示。

从图 8-20 可以看到，控制台输出了指定的消费消息，消费者 consumerTag 的内容为随机生成的；发送消息时没有指定交换机，所以 Exchange 为空；RoutingKey 为 hello_world；文件 8-2 中生成消息时并未指定 props 的内容，所以输出的 properties 中属性值都为空。

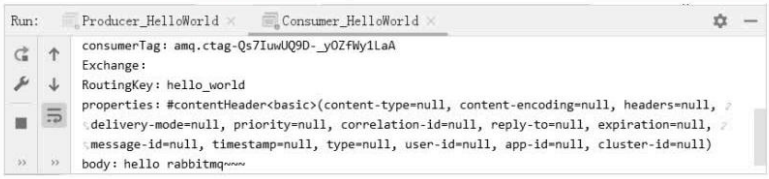


图8-20 Consumer_HelloWorld控制台输出内容

此时再次查看 RabbitMQ 可视化页面 Queues 面板中的内容，如图 8-21 所示。

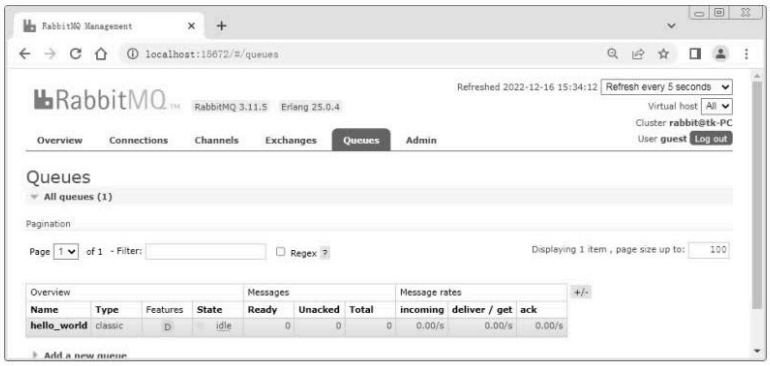


图8-21 Queues面板中的内容（2）

从图 8-21 可以看出，hello_world 队列中已经没有消息了，说明消息成功被消费了。

至此，RabbitMQ 的入门案例已经完成。

8.3　Spring Boot 与 RabbitMQ 整合实现

8.2 节介绍了 RabbitMQ 消息中间件支持的 7 种工作模式，Spring Boot 为 RabbitMQ 的这些工作模式都提供了非常好的整合实现，并集成了多种方式的整合支持，包括基于 API 的方式、基于配置类的方式、基于注解的方式。下面选取常用的 Publish/Subscribe、Routing 和 Topics 这 3 种工作模式完成在 Spring Boot 项目中的消息服务整合实现。

8.3.1　Spring Boot 整合 RabbitMQ 环境搭建

在 Spring Boot 项目中实现基于 RabbitMQ 的消息之前，需要搭建对应的整合环境，下面对 Spring Boot 整合 RabbitMQ 实现消息服务需要的整合环境进行搭建，具体步骤如下。

1. 创建 Spring Boot 项目

使用 Spring Initializr 方式创建一个名称为 rabbitmq 的 Spring Boot 项目，在 Dependencies 依赖选择中选择 Web 模块中的 Spring Web 依赖，以及 Integration 模块中的 RabbitMQ 依赖，如图 8-22 所示。

图8-22　Spring Boot项目依赖选择

2. 编写配置文件

打开创建项目时自动生成的 application.properties 全局配置文件，在该文件中编写 RabbitMQ 服务对应的连接配置，具体如文件 8-4 所示。

文件 8-4　application.properties

```
# 配置 RabbitMQ 消息中间件连接配置
spring.rabbitmq.host=localhost
spring.rabbitmq.port=5672
spring.rabbitmq.username=guest
spring.rabbitmq.password=guest
#配置 RabbitMQ 虚拟主机路径/，默认可以省略
```

```
7 spring.rabbitmq.virtual-host=/
```

在文件 8-4 中，连接的 RabbitMQ 服务端口号为 5672，并使用了默认用户 guest 连接。

小提示：

需要强调的是，在上述项目全局配置文件 application.properties 中，编写了外部 RabbitMQ 消息中间件的连接配置，这样在整合消息服务时，使用的都是自己安装配置的 RabbitMQ 服务。而在 Spring Boot 中，也集成了一个内部默认的 RabbitMQ 中间件，如果没有在配置文件中配置外部 RabbitMQ 连接，会启用内部的 RabbitMQ 中间件，通常建议读者根据当前 RabbitMQ 的情况自行设置配置信息。

8.3.2 使用 Publish/Subscribe 模式实现消息服务

Spring Boot 整合 RabbitMQ 中间件实现消息服务主要围绕三个部分的工作来开展，即定制中间件、消息生产者发送消息、消息消费者接收消息。下面以用户注册成功后同时发送邮件通知和短信通知这一场景为例，分别使用基于API、基于配置类和基于注解这三种方式实现 Publish/ Subscribe 模式的消息服务。

1. 基于 API 的方式

基于 API 的方式主要是指使用 Spring 框架提供的 AMQP 管理类 AmqpAdmin 管理 Exchange、Queue、Binding，这种定制组件的方式与在 RabbitMQ 可视化页面上通过对应面板进行组件操作的实现基本一样，都是预先手动声明交换机、队列、路由键等，然后进行消息队列组装，以供应用程序调用，从而实现消息服务。下面对基于 API 的方式进行讲解和演示。

（1）定制消息组件

Spring 提供了一个操作 RabbitMQ 的消息模板类 RabbitTemplate，该类提供了编辑消息、发送消息、发送消息前的监听、发送消息后的监听等消息制造和消息监听等功能，使用时直接从 Spring 容器中取出即可。

在 rabbitmq 项目中创建名为 com.itheima.rabbitmq.service 的包，在包中创建消息生产服务类 MessageProducerService，在该类的构造方法中使用 AmqpAdmin 管理类定制 Publish/Subscribe 模式所需的组件，具体如文件 8-5 所示。

文件 8-5 MessageProducerService.java

```
1 import com.itheima.rabbitmq.domain.User;
2 import org.springframework.amqp.core.AmqpAdmin;
3 import org.springframework.amqp.core.Binding;
4 import org.springframework.amqp.core.FanoutExchange;
5 import org.springframework.amqp.core.Queue;
6 import org.springframework.amqp.rabbit.core.RabbitTemplate;
7 import org.springframework.beans.factory.annotation.Autowired;
8 import org.springframework.stereotype.Service;
9 @Service
10 public class MessageProducerService {
11     private AmqpAdmin amqpAdmin;
12     private RabbitTemplate rabbitTemplate;
13     @Autowired
14     public MessageProducerService(AmqpAdmin amqpAdmin,RabbitTemplate
rabbitTemplate) {
```

```
 15        this.amqpAdmin=amqpAdmin;
 16        this.rabbitTemplate=rabbitTemplate;
 17        // 1.定义 fanout 类型的交换机
 18        this.amqpAdmin.declareExchange(new
FanoutExchange("pub/sub.exchange"));
 19        // 2.定义两个默认持久化队列，分别处理 email 和 sms
 20        this.amqpAdmin.declareQueue(new Queue("email.queue"));
 21        this.amqpAdmin.declareQueue(new Queue("sms.queue"));
 22        // 3.将队列分别与交换机进行绑定
 23        this.amqpAdmin.declareBinding(new Binding("email.queue",
 24              Binding.DestinationType.QUEUE, "pub/sub.exchange", "", null));
 25        this.amqpAdmin.declareBinding(new Binding("sms.queue",
 26              Binding.DestinationType.QUEUE, "pub/sub.exchange", "", null));
 27    }
 28 }
```

在文件 8-5 中，第 11～12 行代码分别声明了 AmqpAdmin 对象和 RabbitTemplate 对象。第 14～27 行代码定义了 MessageProducerService 类的构造方法，该构造方法中注入 AmqpAdmin 对象和 RabbitTemplate 对象，并创建实现 Publish/Subscribe 模式所需的消息组件，其中，第 18 行代码定义了一个 fanout 类型的交换机 pub/sub.exchange；第 20～21 行代码定义了两个名称分别为 email.queue 和 sms.queue 的队列，分别用来存储邮件信息和短信信息；第 23～26 行代码将定义的两个队列分别与交换机绑定。

（2）实现消息生产

创建实体类。完成消息组件的定制工作后，创建消息生产者并发送消息到消息队列中。在发送消息时，通常会将消息封装在类中进行传递，在此，在 rabbitmq 项目中创建名为 com.itheima.rabbitmq.domain 的包，并在该包下创建一个用户 User，具体如文件 8-6 所示。

文件 8-6　User.java

```
 1 public class User {
 2    private Integer id;
 3    private String username;
 4     ……getter/setter 方法，以及 toString()方法
 5 }
```

创建生产消息的方法。在文件 8-5 中新增发送消息的方法 psubPublisher()，该方法中使用 RabbitTemplate 对象发送消息，代码如下。

```
public void psubPublisher(User user) {
   this.rabbitTemplate.convertAndSend("pub/sub.exchange","",user);
}
```

在上述代码中，使用 RabbitTemplate 对象的 convertAndSend(String exchange, String routingKey, Object object)方法进行消息发布。convertAndSend(String exchange, String routingKey, Object object)方法中的第一个参数表示发送消息的交换机，这个参数值要与之前定制的交换机名称一致；第二个参数表示路由键，因为实现的是 Publish/Subscribe 模式，所以不需要指定；第三个参数是发送的消息内容。

创建控制器。在 rabbitmq 项目中创建名为 com.itheima.rabbitmq.controller 的包，并在该包下创建一个控制器类，在该类中定义方法调用生产消息的方法，具体如文件 8-7 所示。

文件 8-7　UserController.java

```
 1 import com.itheima.rabbitmq.domain.User;
```

```
import com.itheima.rabbitmq.service.MessageProducerService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;
@RestController
public class UserController {
    @Autowired
    private  MessageProducerService messageProducerService;
    @RequestMapping("/user/{id}/{username}")
    public void psubPublisher(@PathVariable Integer id,@PathVariable String
username) {
        User user=new User(id,username);
        messageProducerService.psubPublisher(user);
    }
}
```

在上述代码中，第 9～10 行代码注入了 MessageProducerService 对象，并在第 14 行代码调用 psubPublisher()方法指定生产的具体消息。

指定消息转换器。在消息发送过程中默认使用 SimpleMessageConverter 转换器进行消息转换存储，该转换器只支持 byte（数组）、String（字符串），以及可序列化对象的消息。因此，如果需要发送实体类消息有两种解决方案：第一种方案，直接将实体类实现 JDK 自带的 Serializable 序列化接口；第二种方案，定制其他类型的消息转换器，例如 JSON 格式的消息转换器 Jackson2JsonMessageConverter。这两种实现方案都可行，只不过第一种方案实现后可视化效果很差，转换后的消息无法辨识，因此后续将使用第二种方案，使用较为熟悉的 JSON 格式消息转换器来替换默认转换器。

在 rabbitmq 项目中创建名为 com.itheima.rabbitmq.config 的包，并在该包下创建一个 RabbitMQ 消息配置类 RabbitMQConfig，具体如文件 8-8 所示。

文件 8-8　RabbitMQConfig.java

```
import
org.springframework.amqp.support.converter.Jackson2JsonMessageConverter;
import org.springframework.amqp.support.converter.MessageConverter;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
@Configuration
public class RabbitMQConfig {
    @Bean
    public MessageConverter messageConverter(){
        return new Jackson2JsonMessageConverter();
    }
}
```

在文件 8-8 中，创建了一个配置类 RabbitMQConfig，并在该配置类中通过@Bean 注解自定义了一个 Jackson2JsonMessageConverter 类型的消息转换器组件，本项目消息生产者发送消息时，会使用该类型的消息转换器对消息的格式进行转换。

（3）实现消息消费

在 rabbitmq 项目的 com.itheima.rabbitmq.service 包下创建一个对 RabbitMQ 消息中间件进

行消息接收和处理的服务类 MessageConsumerService，在该类中实现消息的消费，具体如文件 8-9 所示。

文件 8-9　MessageConsumerService.java

```
1  import org.springframework.amqp.core.Message;
2  import org.springframework.amqp.rabbit.annotation.RabbitListener;
3  import org.springframework.stereotype.Service;
4  @Service
5  public class MessageConsumerService {
6      /**
7       * Publish/Subscribe 模式接收，处理邮件业务
8       */
9      @RabbitListener(queues = "email.queue")
10     public void psubConsumerEmail(Message message) {
11         byte[] body = message.getBody();
12         String s = new String(body);
13         System.out.println("邮件业务接收到消息：  "+s);
14
15     }
16     /**
17      * Publish/Subscribe 模式接收，处理短信业务
18      */
19     @RabbitListener(queues = "sms.queue")
20     public void psubConsumerSms(Message message) {
21         byte[] body = message.getBody();
22         String s = new String(body);
23         System.out.println("短信业务接收到消息：  "+s);
24     }
25 }
```

在文件 8-9 中，创建了一个接收处理 RabbitMQ 消息的业务处理类 MessageConsumer Service，在该类中使用 Spring 框架提供的@RabbitListener 注解分别监听名称为 email.queue 和 sms.queue 的队列，监听的这两个队列是前面指定发送并存储消息的消息队列。一旦服务启动且监听到指定的队列中有消息存在，对应注解的方法会立即接收并消费队列中的消息。

需要说明的是，使用@RabbitListener 注解所标注的接收消息的方法中，参数类型可以与发送的消息类型保持一致，或者使用 Object 类型和 Message 类型。如果使用与消息类型对应的参数接收消息的话，只能够得到具体的消息体信息；如果使用 Object 或者 Message 类型参数接收消息的话，还可以获得除了消息体外的消息参数信息 MessageProperties。

（4）测试效果

为了能更好地观察消息的生产和消费情况，在文件 8-9 的第 11 行代码和第 21 行代码打上断点，并以 Debug 模式启动项目。项目启动成功后，通过 RabbitMQ 可视化管理页面查看 Queues 面板，效果如图 8-23 所示。

从图 8-23 可以看出，在 RabbitMQ 可视化管理页面的 Queues 面板中新增了两个队列，名称分别为 email.queue 和 sms.queue，说明使用 AmqpAdmin 成功定制了对应的队列。

单击图 8-23 中的“ Exchanges ”查看 Exchanges 面板，效果如图 8-24 所示。

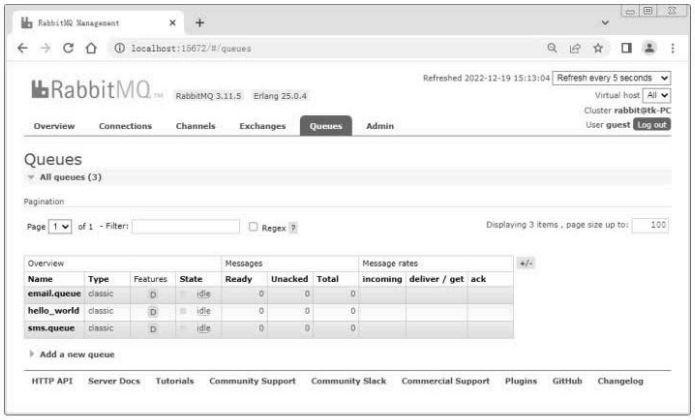


图8-23 Queues面板（1）

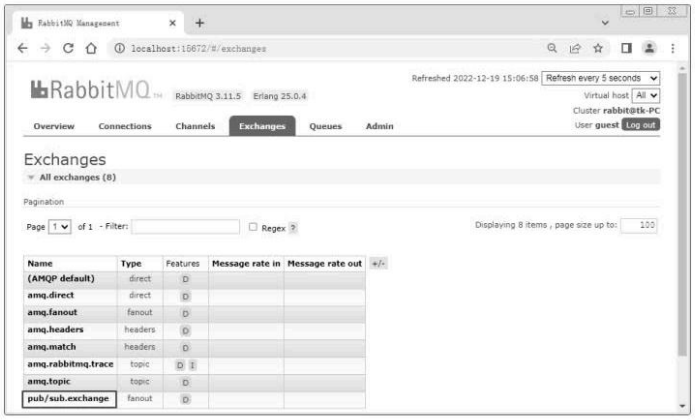


图8-24 Exchanges面板

从图8-24可以看出，在RabbitMQ可视化管理页面的Exchanges面板中有8个Exchanges，其中名称为 pub/sub.exchange、类型为 fanout 的交换机为本案例自定义的交换机，剩余的交换机是 RabbitMQ 自带的，且其类型是系统预先设置好的。

在图 8-24 中单击“ pub/sub.exchange ”，进入 Exchanges 面板查看交换机详情，效果如图 8-25 所示。

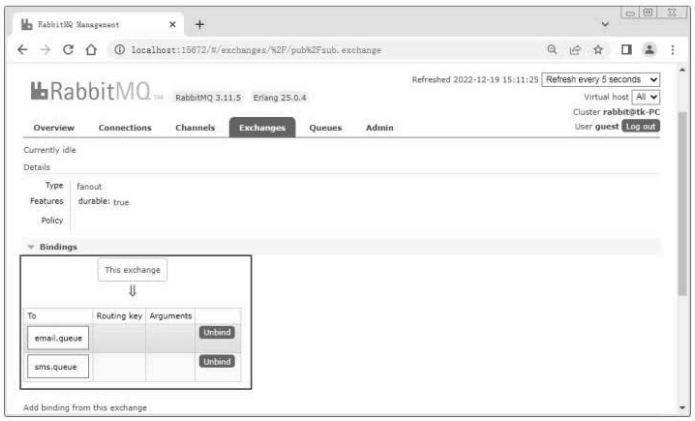


图8-25 Exchanges详情

从图 8-25 可以看出，在 pub/sub.exchange 交换机详情页面中，展示有该交换机的具体信息，还有与之绑定的两个队列 email.queue 和 sms.queue，说明交换机和队列根据自定义的绑定规则进行了绑定。

在浏览器中访问“ http://localhost:8080/user/1/zhangsan ”，此时会触发生产消息的操作。再次查看 Queues 面板，效果如图 8-26 所示。

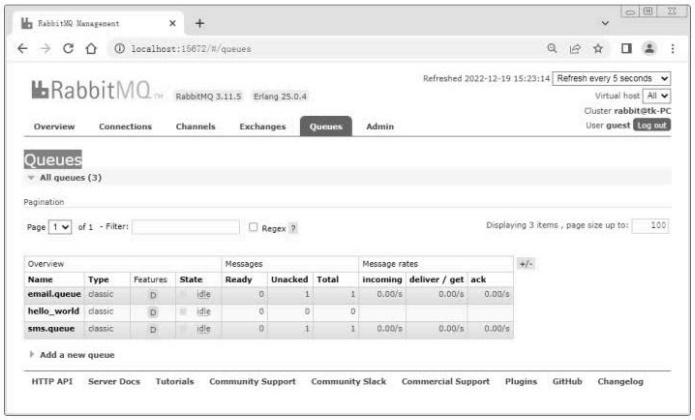


图8-26　Queues面板（2）

从图 8-26 可以看出，触发生产消息的操作后，Publish/Subscribe 模式下绑定的两个队列中各有一条状态为 Unacked（待应答）的消息，代表消息已经投递给消息消费者，但还没有收到消息消费者的反馈。消息待应答是因为之前在消费消息的方法中打了断点，尚未放行。此时，对断点进行放行后，IDEA 控制台输出消息，如图 8-27 所示。

图8-27　控制台输出消息

从图 8-27 可以看出，控制台输出了消息生产者所生产的消息内容，此时再次查看 Queues 面板，效果如图 8-28 所示。

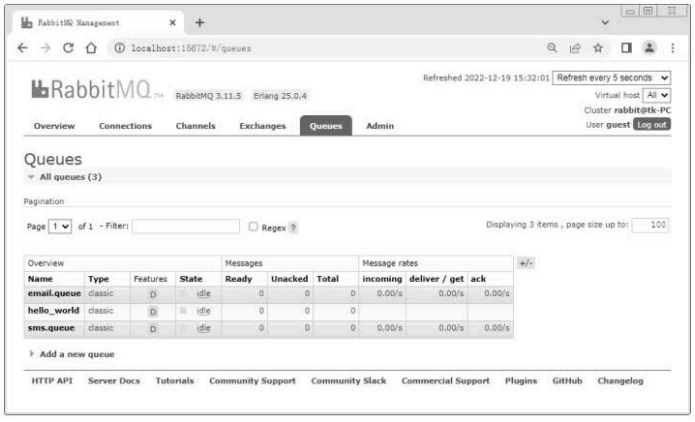


图8-28　Queues面板（3）

从图 8-28 可以看出，email.queue 和 sms.queue 队列中没有消息，说明消息成功被生产和消费。

至此，使用 Publish/Subscribe 模式实现消息服务已经完成。

2. 基于配置类的方式

基于配置类的方式主要是指使用配置类定制消息组件。下面对采用这种方式定制消息组件进行讲解。

（1）定制消息组件

在文件 8-8 的 RabbitMQ 消息配置类 RabbitMQConfig 中，定义消息组件交由 Spring 进行管理，修改后的内容如文件 8-10 所示。

文件 8-10 RabbitMQConfig.java

```
import org.springframework.amqp.core.*;
import org.springframework.amqp.support.converter.Jackson2JsonMessageConverter;
import org.springframework.amqp.support.converter.MessageConverter;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
@Configuration
public class RabbitMQConfig {
    @Bean
    public MessageConverter messageConverter(){
        return new Jackson2JsonMessageConverter();
    }
    // 1.定义 fanout 类型的交换机
    @Bean
    public Exchange pubsubExchange(){
        return ExchangeBuilder.fanoutExchange("pub/sub.exchange").build();
    }
    // 2.定义两个不同名称的消息队列
    @Bean
    public Queue emailQueue(){
        return new Queue("email.queue");
    }
    @Bean
    public Queue ssmQueue(){
        return new Queue("sms.queue");
    }
    // 3.将两个不同名称的消息队列与交换机进行绑定
    @Bean
    public Binding bindingEmail(){
        return
BindingBuilder.bind(emailQueue()).to(pubsubExchange()).with("").noargs();
    }
    @Bean
    public Binding bindingSms(){
        return
         BindingBuilder.bind(ssmQueue()).to(pubsubExchange()).with("").noargs();
    }
}
```

在上述代码中，第 13～36 行代码使用@Bean 注解定制了三种类型的 5 个 Bean 组件，分别为 1 个交换机、2 个队列、2 个消息队列分别与交换机的绑定。这种基于配置类方式定制的消息组件内容与基于 API 方式定制的消息组件内容完全一样，只不过实现方式不同。

（2）修改消息生产服务类

按照消息服务整合实现步骤完成消息组件的定制后，还需要编写消息生产者和消息消费者，而在基于 API 的方式中已经实现了消息生产者和消息消费者。为了验证消息组件是由消息配置类实现的，将文件 8-5 中制定消息组件的构造器删除或注释掉，并注入 RabbitTemplate 对象。文件 8-5 修改后的代码如文件 8-11 所示。

文件 8-11　MessageProducerService.java

```
1  import com.itheima.rabbitmq.domain.User;
2  import org.springframework.amqp.rabbit.core.RabbitTemplate;
3  import org.springframework.beans.factory.annotation.Autowired;
4  import org.springframework.stereotype.Service;
5  @Service
6  public class MessageProducerService {
7      @Autowired
8      private RabbitTemplate rabbitTemplate;
9      public void psubPublisher(User user) {
10         this.rabbitTemplate.convertAndSend("pub/sub.exchange","",user);
11     }
12 }
```

（3）测试效果

重新启动项目，在浏览器中发送请求触发消息生产的操作，消息消费者可以自动监听并消费消息队列中存在的消息，测试效果与基于 API 的方式一样，读者可以自行验证，在此就不再一一展示。

3. 基于注解的方式

基于注解的方式主要是指使用 Spring 框架提供的@RabbitListener 注解及其相关属性定制消息组件，下面对这种基于注解的方式进行讲解。

（1）定制消息组件

在文件 8-10 中注释掉或删除第 13～36 行代码，取消在配置类中定义交换机、队列和这两者之间的绑定。在文件 8-9 的消息消费服务类 MessageConsumerService 的邮件业务和短信业务处理的消息消费方法上，使用@RabbitListener 注解及其相关属性定制消息组件，并消费消息，修改后的内容如文件 8-12 所示。

文件 8-12　MessageConsumerService.java

```
1  import com.itheima.rabbitmq.domain.User;
2  import org.springframework.amqp.rabbit.annotation.Exchange;
3  import org.springframework.amqp.rabbit.annotation.Queue;
4  import org.springframework.amqp.rabbit.annotation.QueueBinding;
5  import org.springframework.amqp.rabbit.annotation.RabbitListener;
6  import org.springframework.stereotype.Service;
7  @Service
8  public class MessageConsumerService {
9      @RabbitListener(bindings =@QueueBinding(value =@Queue("email.queue"),
10             exchange =@Exchange(value = "pub/sub.exchange",type = "fanout")))
```

```
    public void psubConsumerEmail(User user) {
        System.out.println("邮件业务接收到消息:  "+user);

    }
    @RabbitListener(bindings =@QueueBinding(value =@Queue("sms.queue"),
            exchange =@Exchange(value = "pub/sub.exchange",type = "fanout")))
    public void psubConsumerSms(User user) {
        System.out.println("短信业务接收到消息:  "+user);
    }
}
```

在文件 8-12 中，使用@RabbitListener 注解标注消息消费的方法，声明了队列，并将这两者建立绑定关系，若 RabbitMQ 中不存在该绑定所需要的 Queue、Exchange、RouteKey 则自动创建。该方法改用与发送消息对应的实体类 User 作为消息接收参数。在@RabbitListener 注解中，使用 bindings 属性来自动创建并绑定交换机和消息队列组件，在定制交换机时将交换机类型设置为 fanout。另外，bindings 属性的@QueueBinding 注解除了有 value、type 属性外，还有 key 属性用于定制路由键 routingKey，当前发布订阅模式不需要 routingKey。

（2）测试效果

重新启动项目，在浏览器中发送请求触发消息生产的操作，消息消费者可以自动监听并消费消息队列中存在的消息，测试效果也与基于 API 的方式一样。

至此，在 Spring Boot 中完成了使用基于 API、基于配置类和基于注解这三种方式来实现 Publish/Subscribe 工作模式的整合。其中，这三种实现消息服务的方式中，基于 API 的方式相对简单、直观，但容易与业务代码产生耦合；基于配置类的方式组件和业务相对隔离、容易统一管理、符合 Spring Boot 框架思想；基于注解的方式清晰明了、方便各自管理，但也容易与业务代码产生耦合。在实际开发中，使用基于配置类的方式和基于注解的方式定制组件实现消息服务较为常见，使用基于 API 的方式偶尔使用，使用时应根据实际情况进行选择。

8.3.3 使用 Routing 模式实现消息服务

不管使用 RabbitMQ 的哪种模式，都可以使用基于 API、基于配置类和基于注解这三种方式中的任意一种方式创建交换机、队列，并建立这两者之间的绑定关系。下面以不同级别的日志信息采集处理为例，在 Spring Boot 项目中使用基于注解的方式，并使用 Routing 模式实现消息服务。

1. 实现消息生产

在文件 8-11 的 MessageProducerService 类中新增生产消息的方法 routingPublisher()，在该方法中使用 RabbitTemplate 模板类实现 Routing 路由模式下的消息发送，具体代码如下。

```
public void routingPublisher(String level,String msg) {
    this.rabbitTemplate.convertAndSend("routing.exchange","routingkey."+level,ms
g);
}
```

在上述代码中 ,使用 RabbitTemplate 对象的 convertAndSend(String exchange, String routing Key, Object object)方法进行消息发送。该方法的 exchange 参数为交换机名称；routingKey 参数为路由键，在 Routing 工作模式下发送消息时，必须指定路由键参数，该参数要与消息队

列映射的路由键保持一致，否则发送的消息将会丢失；object 参数为需要生产的消息。消息生产者生产的消息根据 routingKey 发送到对应的队列中存储。

在文件 8-7 的 UserController 类中新增方法 routingPublisher()用于触发消息生产，本案例指定传入的日志信息的格式为“日志信息级别&日志信息内容”，具体代码如下。

```
@RequestMapping("/user/{message}")
public void routingPublisher(@PathVariable String message) {
    String[] m = message.split("&");
    messageProducerService.routingPublisher(m[0],m[1]);
}
```

在上述代码中，方法的 message 参数用于接收传入的日志信息，接收到日志信息后根据符号“&”切割出对应的日志级别和日志信息内容，调用 MessageProducerService 对象的 routingPublisher()方法执行消息生产。

2. 实现消息消费

在文件 8-12 的 MessageConsumerService 类中新增消费消息的方法，消费消息的方法使用@RabbitListener 注解进行标注，具体代码如下。

```
@RabbitListener(bindings =@QueueBinding(value =@Queue("routing.error.queue"),
        exchange =@Exchange(value = "routing.exchange",type = "direct"),
        key = "routingkey.error"))
public void routingConsumerError(String message) {
    System.out.println("接收到 error 级别日志消息:  "+message);
}
/**
 *  路由模式消息接收，处理 info、error、warning 级别日志信息
 */
@RabbitListener(bindings =@QueueBinding(value =@Queue("routing.all.queue"),
        exchange =@Exchange(value = "routing.exchange",type = "direct"),
        key = {"routingkey.error","routingkey.info","routingkey.warning"}))
public void routingConsumerAll(String message) {
    System.out.println("接收到 info、error、warning 等级别日志消息:  "+message);
}
```

在上述代码中，在两个消费消息的方法上使用@RabbitListener 注解及其相关属性定制了路由模式下的消息组件。与发布订阅模式下的注解相比，Routing 路由模式下的交换机类型 type 修改为了 direct，并且指定了路由键 routingKey。

3. 测试效果

启动项目后，RabbitMQ 会根据文件 8-12 中@RabbitListener 注解的信息创建对应的交换机、队列，以及交换机和队列的绑定关系。通过 RabbitMQ 可视化管理页面查看 routing.exchange 交换机的详细信息，如图 8-29 所示。

从图 8-29 可以看出，routing.exchange 交换机绑定了名称为 routing.all.queue 和 routing. error.queue 的两个队列，其中队列 routing.all.queue 可以通过 routingkey.error、routingkey.info、routingkey.warning 三个路由键进行匹配；routing.error.queue 可以通过 routingkey.error 路由键匹配。

接着测试发送 error 级别的日志信息。在浏览器中访问“http://localhost:8080/user/error&Out Of Memory”，控制台输出消息如图 8-30 所示。

如图 8-30 所示，控制台输出两条消息消费者获取的消息，说明生产 error 级别的日志信息后，两个消息消费者方法都根据路由键 routingkey.error 对 routing.all.queue 和 routing.error.

queue 队列中的日志信息进行了消费。

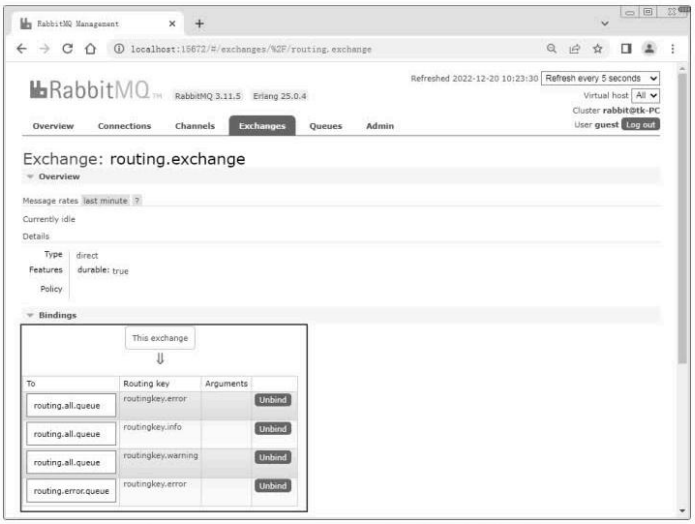


图8-29 routing.exchange交换机的详细信息

最后，测试发送 info 级别的日志信息。在浏览器中访问“ http://localhost:8080/user/info&Tomcat initialized ”，控制台输出消息如图 8-31 所示。

图8-30 Routing模式控制台输出消息（1）

图8-31 Routing模式控制台输出消息（2）

如图 8-31 所示，控制台输出一条消息消费者获取的消息，说明生产 info 级别的日志信息后，只有一个消费消息的方法根据路由键 routingkey.info 对 routing.all.queue 队列中的日志信息进行了消费。

8.3.4 使用 Topics 模式实现消息服务

Topics 模式的功能也很强大，下面以不同用户对邮件和短信的订阅需求不同这一场景为例，在 Spring Boot 项目中使用基于注解的方式，并使用 Topics 模式实现消息服务。

1. 实现消息生产

在文件 8-11 的 MessageProducerService 类中新增生产消息的方法 topicPublisher()，在该方法中使用 RabbitTemplate 模板类实现 Topics 模式下的消息发送，具体代码如下。

```
public void topicPublisher(String routingkey,String msg) {
    this.rabbitTemplate.convertAndSend("topic.exchange",routingkey,msg);
}
```

在上述代码中，使用 RabbitTemplate 对象的 convertAndSend(String exchange, String routingKey, Object object)方法进行消息发送，该方法的 exchange 参数为交换机名称，routingKey 参数为路由键，object 参数为需要生产的消息。消息生产者生产的消息根据 routingKey 发送到对应的队列中存储。

在文件 8-7 的 UserController 类中新增方法 topicPublisher()用于触发消息生产，本案例

指定传入的信息的格式为“路由键&消息内容”，具体代码如下。

```
@RequestMapping("/user/topic/{message}")
public void topicPublisher(@PathVariable String message) {
    String[] m = message.split("&");
    messageProducerService.topicPublisher(m[0],m[1]);
}
```

在上述代码中，方法的 message 参数接收传入的消息，接收到消息后根据符号“&”切割出对应的路由键和消息，调用 MessageProducerService 对象的 topicPublisher()方法执行消息生产。

2. 实现消息消费

在文件 8-12 的 MessageConsumerService 类中新增消费消息的方法，消费消息的方法使用@RabbitListener 注解进行标注，具体代码如下。

```
1  /**
2   *  通配符模式消息接收，进行邮件业务订阅处理
3   */
4  @RabbitListener(bindings =@QueueBinding(value =@Queue("topic.email.queue"),
5          exchange =@Exchange(value = "topic.exchange",type = "topic"),
6          key = "info.#.email.#"))
7  public void topicConsumerEmail(String message) {
8      System.out.println("接收到邮件订阅需求处理消息:  "+message);
9  }
10 /**
11  *  通配符模式消息接收，进行短信业务订阅处理
12  */
13 @RabbitListener(bindings =@QueueBinding(value =@Queue("topic.sms.queue"),
14         exchange =@Exchange(value = "topic.exchange",type = "topic"),
15         key = "info.#.sms.#"))
16 public void topicConsumerSms(String message) {
17     System.out.println("接收到短信订阅需求处理消息:  "+message);
18 }
```

在上述代码中，在两个消费消息的方法上使用@RabbitListener 注解及其相关属性定制了路由模式下的消息组件。与使用 Routing 模式的注解内容相比，Topics 模式的注解使用方式与 Routing 模式的使用方式基本一样，主要是将交换机类型 type 修改为了 topic，还分别使用通配符的样式指定了路由键，并创建了动态路由键。其中，第 6 行代码和第 15 行代码中 key 中的“#”可以匹配 0 至多个字符。例如，路由键 infor.email 可以与 info.#.email.#相匹配，路由键 infor.sms 可以与 info.#.sms.#相匹配，路由键 infor.email.sms 可以同时匹配 info.#.email.#和 info.#.sms.#。

3. 测试效果

启动项目后，RabbitMQ 会根据文件 8-12 中@RabbitListener 的信息创建对应的交换机、队列，以及交换机和队列的绑定关系，通过 RabbitMQ 可视化管理页面查看 topic.exchange 交换机的详细信息，如图 8-32 所示。

从图 8-32 可以看出，topic.exchange 交换机绑定了名称为 topic.email.queue 和 topic.sms.queue 的两个队列。其中，队列 topic.email.queue 可以根据“info.#.email.#”对路由键进行动态匹配；topic.sms.queue 可以根据“info.#.sms.#”对路由键进行动态匹配。

首先，测试只生成邮件订阅消息。在浏览器中访问“ http://localhost:8080/user/topic/info.email&topic email message ”，控制台输出消息如图 8-33 所示。

如图 8-33 所示 ,控制台输出一条消息消费者获取的消息 ,说明指定路由键为 info.email 时，RabbitMQ 根据路由键只执行 topicConsumerEmail()方法 ,消费了 topic.email.queue 队列中的消息。

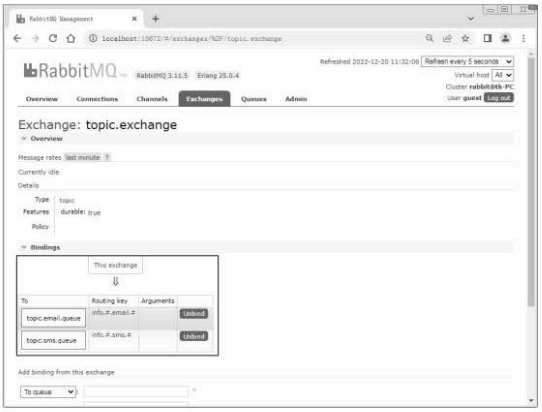


图8-32 topic.exchange交换机的详细信息

图8-33 Topic模式控制台输出消息（1）

然后，测试只生成短信订阅消息。在浏览器中访问“ http://localhost:8080/user/topic/info.sms&topic sms message ”，控制台输出消息如图 8-34 所示。

如图 8-34 所示，控制台输出一条消息消费者获取的消息，说明指定路由键为 info.sms 时，RabbitMQ 根据路由键只执行 topicConsumerSms()方法，消费了 topic.sms.queue 队列中的消息。

最后，测试同时生成邮件订阅消息和短信订阅消息。在浏览器中访问“ http://localhost:8080/user/topic/info.email.sms&topic email and sms message ”,控制台输出消息如图 8-35 所示。

图8-34 Topic模式控制台输出消息（2）

图8-35 Topic模式控制台输出消息（3）

如图 8-35 所示，控制台输出两条消息消费者获取的消息，说明指定路由键为 info.email.sms 时 ,RabbitMQ 根据路由键执行 topicConsumerSms()和 topicConsumerEmail()方法，消费了 topic.sms.queue 和 topic.email.queue 队列中的消息。

8.4 本章小结

本章主要对 Spring Boot 项目中的消息服务进行了讲解。首先讲解了消息服务概述；然后讲解了 RabbitMQ 快速入门；接着讲解了 Spring Boot 与 RabbitMQ 的整合实现。通过本章的学习，希望读者可以在 Spring Boot 项目中正确使用 RabbitMQ 实现消息服务。

8.5　本章习题

一、填空题

1．MQ 是一种能实现_____到消息消费者单向通信的通信模型。

2．消息队列中，_____用于客户端与 RabbitMQ 之间的连接。

3．在 RabbitMQ 的 Publish/Subscribe 模式中，必须先手动配置一个_____类型的交换机。

4．RabbitMQ 服务端口号默认为_____。

5．Spring 框架提供的_____注解可以监听 RabbitMQ 中指定的队列。

二、判断题

1．RabbitMQ 支持的协议有 AMQP。（　　）

2．RabbitMQ 中，一个 Broker 里只能有一个 Virtual Host。（　　）

3．RabbitMQ 的简单模式不用声明交换机，只需要定义一个队列。（　　）

4．安装 RabbitMQ 之前需要先安装 Erlang 语言包。（　　）

5．RabbitTemplate 进行消息发送过程中默认使用 SimpleMessageConverter 转换器进行消息转换存储。（　　）

三、选择题

1．下列选项中，关于 RabbitMQ 的描述正确的是（　　）。

A．一种分布式数据库。　　B．一种集群管理工具。

C．一种消息队列中间件。　　D．一种 Java 安全框架。

2．下列选项中，消息队列中消息的载体是（　　）。

A．Channel　　B．Broker　　C．Exchange　　D．Queue

3．下列选项中，关于消息队列中 Exchange 组件描述正确的是（　　）。

A．消息通道，位于连接内部，负责实际的通信。

B．交换机，用于接收消息生产者发送的消息，并根据分发规则将这些消息路由给服务器中的队列。

C．交换机和消息队列之间的虚拟连接。

D．消息队列服务器实体。

4．下列选项中，关于 RabbitMQ 的模式的描述错误的是（　　）。

A．RabbitMQ 的简单模式中，消息生产者会将消息交给默认的交换机。

B．Work Queues 模式只能有一个消息消费者。

C．Routing 模式需要指定一个 Routing Key。

D．Topics 工作模式适用于根据不同需求动态传递处理业务的场合。

5．下列选项中，Spring 提供的用于管理 Exchange、Queue、Binding 的类是（　　）。

A．AmqpAdmin　　B．RabbitTemplate　　C．Queue　　D．Binding

第9章

任务调度和邮件发送

学习目标

★ 掌握异步任务，能够在 Spring Boot 项目中实现无返回值和有返回值的异步任务
★ 掌握定时任务，能够在 Spring Boot 项目中使用注解的方式实现定时任务
★ 掌握 Quartz 任务调度，能够在 Spring Boot 项目中整合 Quartz 完成任务调度
★ 掌握发送纯文本邮件，能够在 Spring Boot 项目中实现纯文本邮件的发送
★ 掌握发送带附件和图片的邮件，能够在 Spring Boot 项目中实现带附件和图片邮件的发送
★ 掌握发送模板邮件，能够在 Spring Boot 项目中实现模板邮件的发送

开发 Web 应用时，多数应用都具备任务调度功能，使系统可在预定义的时间执行指定的任务，例如系统的负荷凌晨最小，指定每天凌晨执行数据备份的任务。有时候希望任务执行后将结果自动通知给指定的人员，可以在任务执行后将结果以电子邮件的方式进行发送。Spring Boot 为任务调度和邮件发送提供了非常好的支持，下面将介绍 Java 中任务调度和邮件发送的相关知识，并在 Spring Boot 项目中对任务调度和邮件发送进行整合开发。

9.1 任务调度

Java 中的任务调度主要是指基于指定的时间点、间隔时间或者执行次数自动执行任务。在 Spring Boot 程序中可以使用多种方法实现任务调度，最常见的有使用 JDK 的 Timer 对象、Spring Task 和集成 Quartz 三种方式。其中，使用 Timer 时，当任务较多时，该方式的性能相对较差；Spring Task 是指 Spring 提供的任务调度组件，调度的任务包括异步任务和定时任务。下面主要在 Spring Boot 项目中对 Spring Task 的异步任务、定时任务，以及集成 Quartz 实现任务调度进行讲解。

9.1.1 异步任务

Web 应用开发中，大多数情况都是通过同步方式完成数据交互处理的，但是，当处理与第三方系统交互时，容易造成响应迟缓的情况，对此可以使用异步执行的方式解决这个问题。异步任务通常用于耗时较长或者不需要立即得到执行结果的业务，在 Spring 中，可以

使用@Async 注解实现异步任务，被@Async 注解标注的方法称之为异步方法。异步方法在执行的时候，会在独立的线程中执行，调用者无须等待它的完成，即可继续其他的操作。

@Async 注解标注的方法可以没有返回值，如果有返回值，则返回值类型必须为 Future 类型。下面在 Spring Boot 项目中分别针对这两种情况进行讲解。

1．无返回值异步任务调用

在实际开发中，可能会遇到需要向新注册用户发送短信验证码的情况，这时可以考虑使用异步任务调用的方式实现，一方面是因为用户对收到验证码的时效性要求没有那么高，另一方面是因为如果在特定时间范围内没有收到验证码，用户可以再次发送验证码。下面通过在 Spring Boot 项目中调用异步的短信验证码服务的案例，演示无返回值的异步任务调用。

（1）创建 Spring Boot 项目

Spring 框架提供了对异步任务的支持，Spring Boot 框架继承了这一异步任务功能，在 Spring Boot 中整合异步任务时，只需在项目中引入 Web 模块中的 Spring Web 依赖就可以使用这种异步任务功能。使用 Spring Initializr 方式创建一个名为 chapter09 的 Spring Boot 项目，在 Dependencies 依赖中选择 Web 模块中的 Spring Web 依赖。

（2）编写业务层异步方法

在 chapter09 项目中创建名为 com.itheima.chapter09.service 的包，并在该包下创建一个业务实现类 AsyncTaskService，在该类中模拟调用短信验证码服务，具体如文件 9-1 所示。

文件 9-1　AsyncTaskService.java

```
1  import org.springframework.scheduling.annotation.Async;
2  import org.springframework.stereotype.Service;
3  import java.time.LocalDateTime;
4  @Service
5  public class AsyncTaskService {
6      @Async
7      public void sendSMS() throws InterruptedException {
8          System.out.println(LocalDateTime.now().withNano(0) +" 开始调用短信验证码业务方法");
9          Long startTime = System.currentTimeMillis();
10         Thread.sleep(5000);
11         Long endTime = System.currentTimeMillis();
12         System.out.println(LocalDateTime.now().withNano(0) +" 短信验证码业务方法执行完毕");
13         System.out.println("执行短信业务方法总耗时：" + (endTime - startTime)+"毫秒");
14     }
15 }
```

在文件 9-1 的 sendSMS()方法中模拟了调用短信服务的短信验证码业务方法。其中，第 6 行代码在方法上方使用了 Spring 框架提供的@Async 注解，表明当前方法是一个异步方法；第 10 行代码通过休眠的方式模拟了调用短信验证码业务方法的过程，并在调用前后记录调用短信验证码业务方法的开始时间、结束时间，以及执行短信业务方法总耗时。

（3）编写控制层业务调用代码

在 chapter09 项目中创建名为 com.itheima.chapter09.controller 的包，在该包下创建控制器类 TaskController，并在该类中定义方法调用 Service 层的短信验证码服务，具体如文件

9-2 所示。

文件 9-2 TaskController.java

```
1  import com.itheima.chapter09.service.AsyncTaskService;
2  import org.springframework.beans.factory.annotation.Autowired;
3  import org.springframework.web.bind.annotation.RequestMapping;
4  import org.springframework.web.bind.annotation.RestController;
5  import java.time.LocalDateTime;
6  @RestController
7  public class TaskController{
8      @Autowired
9      private AsyncTaskService asyncTaskService;
10     @RequestMapping("/sendSMS")
11     public String sendSMS() throws InterruptedException {
12         asyncTaskService.sendSMS();
13         return LocalDateTime.now().withNano(0)+" 成功调用短信验证码服务";
14     }
15 }
```

在文件 9-2 中，第 11～14 行代码定义的 sendSMS()方法用于处理调用短信验证码的请求，其中，第 12 行代码通过 AsyncTaskService 对象调用异步方法 sendSMS()，调用完异步方法后将当前时间作为返回值对请求进行响应。

（4）开启基于注解的异步任务支持

@Async 注解只是标注方法具有异步执行的功能，但异步任务的支持默认情况下并没有开启，Spring 提供的@EnableAsync 注解可以开启基于注解的异步任务支持。在项目启动类上添加@EnableAsync 注解开启异步任务支持，具体如文件 9-3 所示。

文件 9-3 Chapter09Application.java

```
1  import org.springframework.boot.SpringApplication;
2  import org.springframework.boot.autoconfigure.SpringBootApplication;
3  import org.springframework.scheduling.annotation.EnableAsync;
4  @EnableAsync      // 开启基于注解的异步任务支持
5  @SpringBootApplication
6  public class Chapter09Application {
7      public static void main(String[] args) {
8          SpringApplication.run(Chapter09Application.class, args);
9      }
10 }
```

（5）测试效果

启动 chapter09 项目，项目启动成功后，在浏览器中访问“ http://localhost:8080/sendSMS ”，请求调用短信验证码服务，此时浏览器效果如图 9-1 所示。

从图 9-1 可以看出，发送的请求成功调用了短信验证码服务。

IDEA 控制台输出短信验证码业务方法执行的信息，如图 9-2 所示。

图9-1 请求调用短信验证码服务

图9-2 短信验证码业务方法执行的信息

从图 9-2 可以看出，短信验证码业务方法执行结束的时间为“ 2022-12-21T11:06:04 ”，比图 9-1 中响应请求的时间“ 2022-12-21T11:05:59 ”晚了 5 秒，说明控制层调用 sendSMS()方法后，继续向下执行，直接向页面响应结果，而没有等到 sendSMS()方法执行完毕后再对请求进行响应。调用的异步方法会作为一个子线程单独执行，任务通过异步的方式执行成功。

2. 有返回值异步任务调用

在实际开发中，项目中可能会涉及有返回值的异步任务调用。例如，分别统计两个地区的销售金额，最后将销售金额进行累加汇总。下面通过在 Spring Boot 项目中统计两个地区的销售金额的案例，演示有返回值的异步任务调用。

（1）编写异步方法

在文件 9-1 的 AsyncTaskService 类中新增统计地区销售金额的异步方法，具体代码如下。

```
1  @Async
2  public Future<Integer> salesStatistics(String area) throws
InterruptedException {
3      System.out.println(LocalDateTime.now().withNano(0) +" 开始统计【"+area+"】
的销售额");
4      Long startTime = System.currentTimeMillis();
5      int time=new Random().nextInt(5)+1;
6      Thread.sleep(time*1000);
7      Long endTime = System.currentTimeMillis();
8      System.out.println(LocalDateTime.now().withNano(0) +area+" 地区销售额统计完
毕");
9      int m=time+1000;
10     System.out.println(area+"销售："+m+"元，统计总耗时：" + (endTime - startTime)+"
毫秒");
11     return new AsyncResult<Integer>(m);
12 }
```

在上述代码中，salesStatistics()方法用于模拟统计所要统计地区的销售金额。其中，第 1 行代码使用@Async 注解标记当前方法为异步方法；第 5～6 行代码获取随机数来模拟统计地区销售金额所消耗的时间，并在模拟统计销售金额的前后记录对应的时间。最后，将模拟的统计金额作为参数传入 AsyncResult 对象中返回。AsyncResult 类为 Future 接口的实现类，异步方法通常使用 AsyncResult 封装异步委托上的异步操作的结果。

（2）编写控制层业务调用代码

在文件 9-2 的 TaskController 异步任务业务处理类中，新增处理统计地区销售金额请求方法，在该方法中调用业务层的异步方法进行统计，具体代码如下。

```
1  @RequestMapping("/statistics")
2  public String salesStatistics() throws Exception {
3      Long st= System.currentTimeMillis();
4      //统计北京地区的销售金额
5      Future<Integer> f1 = AsyncTaskService.salesStatistics("北京");
6      //统计上海地区的销售金额
7      Future<Integer> f2 = AsyncTaskService.salesStatistics("上海");
8      //获取统计结果，并累加
9      Integer result = f1.get()+f2.get();
10     Long et= System.currentTimeMillis();
11     System.out.println("统计总计："+result+"元,控制层调用总耗时："+(et-st)+"毫秒");
12     return LocalDateTime.now().withNano(0) + " 成功执行销售金额统计";
```

```
13 }
```

在上述代码中，salesStatistics()方法处理映射路径为“ /statistics ”的统计地区销售金额的请求。其中，第 5 行代码和第 7 行代码都调用统计地区销售金额的异步方法；第 9 行代码中，f1.get()和 f2.get()方法分别获取了第 5 行代码和第 7 行代码调用异步方法的执行结果；第 11 行代码计算出第 5～10 行代码执行所消耗的时间。

（3）测试效果

启动 chapter09 项目 ,项目启动成功后 ,在浏览器中访问“ http://localhost:8080/statistics ”，发送统计地区销售金额的请求。因为业务层设置了休眠时间，所以浏览器在反应一段时间后才会获取到对应的响应结果，如图 9-3 所示。

从图 9-3 可以看出，发送的请求成功执行了销售金额统计。

IDEA 控制台中统计地区销售金额输出的信息如图 9-4 所示。

图9-3　统计地区销售金额

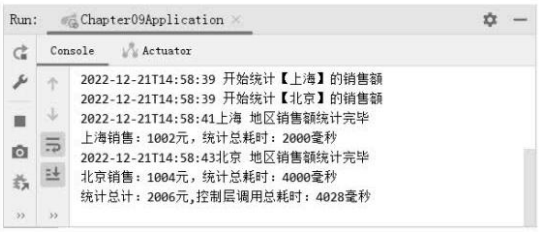


图9-4　统计地区销售金额输出的信息

从图 9-4 可以看出，北京地区的销售额统计和上海地区的销售额统计时间都在“ 2022-12- 21T14:58:39 ”开始，上海地区统计耗时 2 秒，北京地区统计耗时 4 秒，北京地区统计结束时间比上海地区要晚，为“ 2022-12-21T14:58:43 ”,控制层调用统计地区销售金额的方法总耗时为 4028 毫秒，页面获取到的响应结果与北京统计结束时间一样，说明获取到异步方法返回值之前，控制层没有直接响应结果到页面，而是等异步方法执行完毕且返回结果之后，才将结果响应到页面。

使用有返回值的异步任务调用，在执行异步方法时会有短暂阻塞，需要等待并获取异步方法的返回结果，而调用两个异步方法时，程序会使用两个子线程并行执行，直到最后一个异步方法返回结果后才会跳出阻塞状态。

9.1.2　定时任务

定时任务是指按照指定时间周期运行的任务。在实际开发中，当需要在每天的某个固定时间或者每隔一段时间让程序去执行某个任务时，就可以使用定时任务实现，例如，系统根据员工的入职日期自动发送关怀邮件、服务器数据定时在每天零点进行备份等。下面将基于 Spring 的定时任务调度功能对定时任务进行讲解。

1. 定时任务注解

Spring 框架的定时任务调度功能支持配置和注解两种方式 ,Spring Boot 在 Spring 框架的基础上实现了继承 ,并对其中基于注解方式的定时任务实现了非常好的支持。下面对 Spring Boot 项目中基于注解方式的相关注解进行介绍。

（1）@EnableScheduling

@EnableScheduling 注解用于开启基于注解方式的定时任务支持，该注解主要用在项目

启动类上。

（2）@Scheduled

@Scheduled 注解是 Spring 框架提供的定时任务控制的注解，主要用在定时业务方法上。@Scheduled 注解提供了多个属性，可以精细化配置定时任务执行规则，@Scheduled 注解的属性如表 9-1 所示。

表 9-1　@Scheduled 注解的属性

属性	说明
cron	用于指定任务执行的时间规则，为特殊格式的字符串，可以定制定时任务触发的秒、分、小时、日、月、周
zone	用于指定解析 cron 参数时参照的时区。属性值为 String 类型，默认值为空字符串，即使用服务器的本地时区
fixedDelay	一个以毫秒为单位的时间间隔，用于指定上一次任务执行结束后再次执行下一次任务的时间间隔，属性值为 long 类型
fixedDelayString	作用与 fixedDelay 相同，不过属性值为字符串形式的数值
fixedRate	一个以毫秒为单位的时间间隔，用于指定每隔多久执行一次任务，属性值为 long 类型
fixedRateString	作用与 fixedRate 相同，不过属性值为字符串形式的数值
initialDelay	指定第一次执行之前要延迟的毫秒数，需要指定任务执行的规则（fixedRate、fixedRateString、fixedDelay、fixedDelayString），属性值为 long 类型
initialDelayString	作用与 initialDelay 相同，不过属性值为字符串形式的数值

下面分别对表 9-1 中的@Scheduled 注解属性进行讲解并举例说明。

cron 属性

cron 属性是@Scheduled 定时任务注解中最常用也是最复杂的一个属性，其属性值由类似于 cron 表达式的 6 位数组成，可以详细地指定定时任务执行的秒、分、小时、日、月、周，其属性值定义格式如下。

```
"秒 分 小时 日 月 周"
```

从上述格式可知，cron 属性值所代表的时间规则由 6 个域组成，每个域之间使用空格进行分隔。cron 属性值的每个域的取值都比较丰富，具体如表 9-2 所示。

表 9-2　cron 属性值的取值

域	可取值	通配符
秒	0～59	, - * /
分	0～59	, - * /
小时	0～23	, - * /
日	1～31	, - * / ? L
月	1～12、月份对应英文前三个字母，大小写均可	, - * /
周	1～7、星期对应英文前三个字母，大小写均可	, - * / ? L #

表 9-2 中列举了@Scheduled 注解中 cron 属性的值除了可以是基本的数字外，还可以使用一些特殊字符表示的通配符。下面对 cron 属性值支持的通配符所表示的意义进行说明，具体如表 9-3 所示。

表 9-3 cron 属性值支持的通配符

特殊字符	说明	示例
*	表示匹配该域的任意值	@Scheduled(cron = "* * 12 * * ?")表示每天中午 12 点执行一次任务
?	表示不指定值，只能用在月和周两个域，为了避免月和周的冲突	@Scheduled(cron = "0 * * 26 * ?")表示每月的 26 日每分钟执行一次任务
,	表示枚举	@Scheduled(cron = "1,3,5 * * * * ?")表示每天每分钟的第 1、3、5 秒都会执行一次任务
/	表示步长	@Scheduled(cron = "0 */1 * * * ?")表示每隔 1 分钟执行一次任务
-	表示区间	@Scheduled(cron = "0 0 9-18 * * ?")表示每天 9～18 点的整点都执行一次任务
L	表示最后，是单词 Last 的缩写，只能用在日和周两个域	@Scheduled(cron = "0 0 * L * ?")表示每月最后一日每小时执行一次任务
#	表示当前月的第几个周几，只能用在周域	@Scheduled(cron = "0 * * ? * 4#2")表示当前月第四周的周二每分钟执行一次任务

需要说明的是，@Scheduled 注解的 cron 属性的取值类似于 cron 的表达式，但不是完全一致的，cron 属性值只提供 6 位字段赋值。

zone 属性

zone 属性主要与 cron 属性配合使用，用于指定 cron 属性值的时区。在通常情况下，不指定 zone 属性，cron 属性值会自动以服务器所在区域作为本地时区进行表达式解析。例如，中国地区服务器的时区通常默认为 Asia/Shanghai。

fixedDelay 和 fixedDelayString 属性

fixedDelay 和 fixedDelayString 属性的作用相同，都可以指定上一次任务执行结束后再次执行下一次任务的时间间隔，两者的主要区别是属性值的类型不同。其中，fixedDelay 属性值为 long 类型，而 fixedDelayString 属性值为字符串类型，且该字符串必须要能转换成 long 类型。下面通过一个具体的示例来说明，示例代码如下。

```
@Scheduled(fixedDelay = 5000)
@Scheduled(fixedDelayString = "5000")
```

在上述代码中，fixedDelay = 5000 和 fixedDelayString = "5000"都表示在程序启动后，会立即执行一次定时任务，然后在任务执行结束后，每隔 5000 毫秒重复执行一次任务。

fixedRate 和 fixedRateString 属性

fixedRate 和 fixedRateString 属性的作用相同，都可以指定每隔多久执行一次任务，两者的主要区别是属性值的类型不同。其中，fixedRate 属性值为 long 类型，fixedRateString 属性值为字符串类型，且该字符串必须要能转换成 long 类型。下面通过一个具体的示例来说明，示例代码如下。

```
@Scheduled(fixedRate = 5000)
@Scheduled(fixedRateString = "5000")
```

在上述代码中，fixedRate = 5000 和 fixedRateString = "5000"都表示在程序启动后，会立即执行一次定时任务，然后每隔 5000 毫秒重复执行定时任务。如果在单线程中使用这两个属性，应避免线程的阻塞。例如，任务执行所消耗的时间是 7000 毫秒，但是使用 fixedRate

属性或者 fixedRateString 属性设置了每隔 5000 毫秒就执行一次当前程序，那么当过了 5000 毫秒后，Spring 开始调用这个任务时发现当前程序还在执行，就会造成阻塞。

需要说明的是，fixedRate/fixedRateString 属性与 fixedDelay/fixedDelayString 属性的作用有些类似，都是隔一段时间再重复执行定时任务，它们主要区别是：fixedDelay 和 fixedDelayString 属性的下一次执行时间是从上一次任务执行完成后开始计时的；fixedRate 和 fixedRateString 属性的下一次执行时间是从上一次任务执行就开始计时，如果遇到配置的间隔时间小于定时任务执行时间的情况，则下一次任务会在上一次任务执行完成后立即重复执行。

initialDelay 和 initialDelayString 属性

initialDelay 和 initialDelayString 属性的作用相同，需要与 fixedRate、fixedRateString、fixedDelay、fixedDelayString 属性配合使用，指定定时任务第一次执行的延迟时间，然后再按照各自相隔时间重复执行任务。下面通过一个具体的示例来说明，示例代码如下。

```
@Scheduled(initialDelay=1000, fixedDelay=5000)
```

上述代码表示在程序启动后，会延迟 1000 毫秒后再执行第一次任务，然后每隔 5000 毫秒重复执行任务。

2. 定时任务演示

下面在 Spring Boot 实现一个简单的定时任务案例，演示定时任务的使用。

（1）编写定时任务业务处理方法

在 chapter09 项目的 com.itheima.chapter09.service 的包下新建一个定时任务管理的业务处理类 ScheduledTaskService，并在该类中编写对应的定时任务处理方法，具体如文件 9-4 所示。

文件 9-4　ScheduledTaskService.java

```
 1  import org.springframework.scheduling.annotation.Scheduled;
 2  import org.springframework.stereotype.Service;
 3  import java.time.LocalDateTime;
 4  import java.time.format.DateTimeFormatter;
 5  @Service
 6  public class ScheduledTaskService {
 7      DateTimeFormatter fm = DateTimeFormatter.ofPattern("yyyy 年 MM 月 dd 日 HH:mm:
ss");
 8          private Integer count1 = 1;
 9          private Integer count2 = 1;
10          private Integer count3 = 1;
11          @Scheduled(fixedRate = 2000)
12          public void fixedRateScheduledTask() {
13              System.out.println("【1】fixedRate 第"+count1+"次执行，当前时间为："
14                     + LocalDateTime.now().format(fm));
15              count1++;
16          }
17          @Scheduled(fixedDelay = 1000)
18          public void fixedDelayScheduledTask() throws InterruptedException {
19             System.out.println("【2】fixedDelay 第"+count2+"次执行，当前时间为："
20                    + LocalDateTime.now().format(fm));
21              count2++;
22             Thread.sleep(2000);
23          }
```

```
24      @Scheduled(cron = "* * * * * ?")
25      public void cronScheduledTask(){
26          System.out.println("【3】cron 第"+count3+"次执行，当前时间为："
27                  + LocalDateTime.now().format(fm));
28          count3++;
29      }
30 }
```

在文件 9-4 中，使用@Scheduled 注解声明了三个定时任务方法。其中，第 11 行代码使用@Scheduled 注解的 fixedRate 属性指定每隔 2000 毫秒执行一次 fixedRateScheduledTask()方法；第 17 行代码使用@Scheduled 注解的 fixedDelay 属性指定执行 fixedDelayScheduledTask()方法后，每隔 1000 毫秒再次执行，其中 fixedDelayScheduledTask()方法执行时会休眠 2000 毫秒；第 24 行代码使用@Scheduled 注解的 cron 属性指定 cronScheduledTask()方法每隔 1 秒执行一次。

（2）开启基于注解的定时任务支持

为了使 Spring Boot 中基于注解方式的定时任务生效，在项目启动类上使用@EnableScheduling 注解开启基于注解的定时任务支持，具体如文件 9-5 所示。

文件 9-5　Chapter09Application.java

```
1  import org.springframework.boot.SpringApplication;
2  import org.springframework.boot.autoconfigure.SpringBootApplication;
3  import org.springframework.scheduling.annotation.EnableAsync;
4  import org.springframework.scheduling.annotation.EnableScheduling;
5  @EnableScheduling    // 开启基于注解的定时任务支持
6  @EnableAsync         // 开启基于注解的异步任务支持
7  @SpringBootApplication
8  public class Chapter09Application {
9      public static void main(String[] args) {
10         SpringApplication.run(Chapter09Application.class, args);
11     }
12 }
```

（3）测试效果

启动 chapter09 项目，项目启动后控制台输出信息如图 9-5 所示。

从图 9-5 可以看出，项目启动成功后，配置@Scheduled 注解的 fixedRate 和 fixedDelay 属性的定时方法会立即执行一次。然后，@Scheduled 注解中配置 fixedRate 属性的方法每隔 2 秒执行一次；@Scheduled 注解中配置 fixedDelay 属性的方法会在方法执行完成后，再隔 1 秒执行一次，由于方法执行时休眠了 2 秒，所以下次执行方法的时间比上一次执行的时间晚 3 秒；@Scheduled 注解中配置 cron 属性的方法根据表达式每隔 1 秒执行一次。

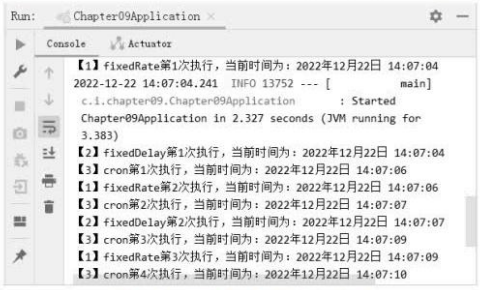


图9-5　定时任务调用效果